11 4 15 Workable Items And Tracking

§ . . — Item-status tracking mandate

Every active item tracked in the project's Issues file MUST carry a `**Status:**` line within five lines of its heading. The status value is drawn from a closed set:

State	Meaning
`Queued`	In backlog, no work has started. Sub-state qualifier `Queued — BLOCKED on <reason>` permitted.
`In progress`	Active investigation or coding underway.
`Ready for testing`	Source-side fix landed (pre-build / meta-test gates green), waiting for the next deploy cycle.
`In testing`	A deploy + cycle is in flight that exercises the fix; live captured-evidence is being collected per §11.4.2 + §11.4.5.
`Reopened`	Item failed its runtime test cycle and is back to active work. Carries a reference to the failure-cycle artefact.
`Fixed (→ Fixed file)`	Item closed: captured-evidence per §11.4.5 collected, runtime test PASSes, paired meta-test mutation FAILs, item migrated to the Fixed file.

Status MUST be updated as the item progresses. Every commit that changes an item's lifecycle state MUST update its Status line in the same commit.

All three Issues / Issues_Summary / Fixed file types MUST be in sync at all times (Markdown + HTML + PDF).

§ . . — Item-type tracking mandate

Every active item tracked in the project's Issues file MUST carry a `**Type:**` line within eight non-blank lines of its heading (the same window the §11.4.15 Status audit uses). The type value is drawn from a CLOSED set of three values:

Type	Meaning
`Bug`	Product defect / regression / user-visible broken behaviour. The product worked before (or was expected to) and now does NOT for the end user.
`Feature`	New capability not previously offered to end users — a new integration, new output, new probe, new bank, new architectural surface.
`Task`	Internal workstream — not user-visible. Refactor, infrastructure, documentation, audit, covenant clause, gate, mutation, propagation enforcement, host-session-safety hardening. The largest class by count; the lowest-stakes default when the classification is ambiguous.

The vocabulary is CLOSED — only `Bug`, `Feature`, `Task`. Any other value is a violation. When ambiguous, fall back to `Task`.

Type tells the operator WHAT KIND of work an item is — distinct from severity (HOW URGENT) and status (WHERE IN LIFECYCLE). The three axes together drive ownership routing, testing strategy, release-note framing, and changelog classification.

The Issues_Summary file MUST carry the Type column for every active item. All three file types (Markdown + HTML + PDF) MUST stay in sync under the same `CM-DOCS-EXPORT-SYNC` discipline as §11.4.12 + §11.4.15.

Pre-build gates `CM-ITEM-TYPE-TRACKING` (scans Issues file for `**Type:**` presence within the audit window) and `CM-COVENANT-114-16-PROPAGATION` (anchor presence in every project CLAUDE.md / AGENTS.md) enforce the mandate. Paired mutations in the project's meta-test prove neither gate is a bluff gate.

No escape hatch — items that genuinely don't fit MUST be re-classified as `INFORMATIONAL` and excluded from the active-item set rather than carry a fabricated Type.

§ . . — Operator-blocked status + self-resolution exhaustion mandate (User mandate, - -)

Forensic anchor — direct user mandate (verbatim, 2026-05-14):

> "Add into Issues and Issues_Summary new status: Operator blocked! We need another column with details in Issues document with exact details on how exactly is operator blocked issue! Before issue becomes operator blocked we MUST investigate in-depth all ways on how it can be resolved without operator's involvement - by you, or by activating any relevant mechanism for the resolution!"

Every project that maintains an Issues / open-work tracker MUST treat operator dependency as a **last-resort classification**, earned only after documented exhaustion of self-resolution paths. Routing an item to the operator without proving the agent + tooling cannot unblock it themselves is the §11.4 PASS-bluff pattern transplanted into the planning layer — the work stalls indefinitely while the agent claims "nothing more I can do."

1. Status vocabulary extension. §11.4.15's closed-set of Status values is extended with a seventh value, `Operator-blocked`. The full closed-set becomes:

#	Status value	Meaning
1	`Queued`	Awaiting work to start.
2	`In progress`	Active agent / developer work.
3	`Ready for test-ing`	Source-side fix landed, awaiting verification.
4	`In testing`	Captured-evidence cycle running.
5	`Reopened`	Previously closed item resurfaced (regression).
6	`Operator-blocked`	**NEW.** Cannot proceed without an operator action that the agent + available tooling cannot perform.
7	`Fixed (→ Fixed.md)`	Closure migration marker per §11.4.15 + §11.4.19.

2. Self-resolution exhaustion mandate. BEFORE classifying any item as `Operator-blocked`, the agent MUST verifiably exhaust every self-resolution path applicable to the project:

- **(a) CLI / ADB / SSH / API access** — can the work be done via any access the agent already has? (e.g. `adb shell pm clear`, `gh api`, `aws ssm`, `kubectl exec`, `psql`).
- **(b) Subagent delegation** — can a specialised subagent (per §11.4.20) reach further than the primary agent? (e.g. `general-purpose` for multi-step probing, `code-reviewer` for architectural blockers, `iac-reviewer` for infra access).
- **(c) Existing tooling** — is there a script / helper / library already in the repo that handles this case? (e.g. `scripts/testing/<helper>.sh`, project-specific automation).
- **(d) Captured fallback** — can the blocker be sidestepped with a synthetic event, test asset substitution, mock, or topology-aware SKIP per §11.4.3? (e.g. simulate the input, substitute a stand-in fixture, downgrade to a pre-build gate).
- **(e) Documentation + research** — per §11.4.8, has the agent consulted external sources (official docs, vendor guides, open-source codebases, issue trackers) for a self-resolution pattern the community has already solved?

Only AFTER each applicable path is **verifiably attempted and documented as exhausted** is `Operator-blocked` the correct classification. "I didn't try X because I assumed it wouldn't work" is a §11.4.6 no-guessing violation AND a §11.4.21 self-resolution violation.

3. Operator-Block-Details mandate. Every `Status: Operator-blocked` item MUST carry an `**Operator-Block-Details:**` line within 8 non-blank lines of its heading (mirroring the §11.4.15 Status placement pattern). The content MUST state ALL of:

- **WHAT** — the specific concrete action the operator must perform (verb + object + target system). Generic "the operator must investigate" is forbidden — name the action.
- **WHY** — the alternatives that were exhausted, enumerated. Each exhausted path from §11.4.21.2 above gets a one-line statement of what was attempted and why it could not unblock the item. "Subagent delegation: tried general-purpose with X scope — blocked because Y" is the bar.
- **UNBLOCK CONDITION** — the observable signal that the operator has completed the action (e.g. "operator confirms hardware rev-B installed", "operator pastes new API key into `scripts/testing/secrets/.foo.env`", "operator force-pushes PR #123 merge to upstream"). Without this, the agent cannot automatically detect when to re-evaluate.
- **WHO** — the handle / contact / pointer to the document where operator-side details live if questions arise (e.g. "operator: @username", "see `docs/operator/<topic>.md`", "vendor: support-id 12345"). Operator-blocked without WHO is operationally untrackable.

4. Issues_Summary inclusion as sortable axis. The auto-generated `Issues_Summary.md` (per §11.4.12 + §11.4.15) MUST include `Operator-blocked` as a first-class Status value in the table — the column header is unchanged but operators can sort / filter on it deterministically. The summary generator MUST also emit a count of `Operator-blocked` items in any rollup line / header.

5. Periodic re-evaluation requirement. Items in `Operator-blocked` status MUST be re-evaluated every Nth tag-cycle (project-defined, recommended every 3rd tag cycle). Operator dependencies change over time: CI may have been added, hardware may have been swapped, automation may have matured, vendor APIs

may have unlocked. An item that was correctly Operator-blocked at tag T may be self-resolvable at tag T+3. Re-evaluation MUST follow the same §11.4.21.2 exhaustion checklist and document each re-attempt.

6. Anti-bluff layer. A fake `Operator-blocked` (classification applied without exhausting §11.4.21.2 alternatives) is a §11.4 covenant violation at the planning layer — equivalent severity to a PASS-bluff at the testing layer. The agent's commit message introducing or maintaining an `Operator-blocked` classification MUST contain evidence of self-resolution attempts (the explicit "Attempted: a — exhausted because X; b — exhausted because Y; c — exhausted because Z" form).

7. Pre-build gates (recommended, per consuming project):

- **`CM-ITEM-OPERATOR-BLOCKED-DETAILS`** — every heading whose Status line equals `Operator-blocked` has an `**Operator-Block-Details:**` line within 8 non-blank lines. Paired mutation strips the details line → gate FAILs.
- **`CM-OPERATOR-BLOCKED-SELF-RESOLUTION-AUDIT`** — every NEW `Operator-blocked` item introduced in a commit carries an "Attempted: ..." audit trail (either in the Issues.md entry body OR in the commit message). Paired mutation introduces an `Operator-blocked` item without the audit trail → gate FAILs.

Propagation. This anchor is a §11.4.17-classified **universal** rule — it composes with §11.4.15 (Status tracking), §11.4.16 (Type tracking), §11.4.19 (Fixed-document column alignment), §11.4.12 (auto-generated docs sync), §11.4.20 (subagent delegation as a self-resolution path), §11.4.8 (research-before-implementation as a self-resolution path), §11.4.6 (no guessing about what the operator "would have to do"). Propagation gate `CM-COVENANT-114-21-PROPAGATION` (when implemented per consuming project) enforces the anchor's presence in every CLAUDE.md / AGENTS.md across the parent + every owned submodule + every dependency.

No escape hatch. Items that legitimately need operator intervention are real and unavoidable — hardware swaps, vendor contracts, physical-world inputs, account credentials, irreversible business decisions. But the classification must be **earned** via documented exhaustion. An `Operator-blocked` heading without an audit trail of self-resolution attempts is a §11.4 release blocker regardless of context.

---

§ . . — Type-aware closure-status vocabulary (User mandate, - -)

Forensic anchor — direct user mandate (verbatim, 2026-05-15):

> "make sure we use proper wording for the workable item completion status in Issues, Issues_Summary and Fixed docs: - Fixed -> into the proper wording depending on the workable item type (task, feature, and so on) - for example: Fixed, Implemented, Completed. Add this important detail in our root Constitution, CLAUDE.MD and AGENTS.MD so it is ALWAYS respected and followed!"

Classification: §11.4.17-classified **universal** — naming discipline applies to every project that tracks work items by type. Bug closures aren't "implemented", feature deliveries aren't "fixed", and infrastructure refactors aren't either. Type-mismatched closure vocabulary is a quiet but persistent semantic-drift bluff at the documentation layer — it makes greppable releases lie about what shipped.

The mandate. §11.4.15 defined the lifecycle Status closed-set including the closure terminal value `Fixed (→ Fixed.md)`. §11.4.16 defined the Type closed-set `{Bug | Feature | Task}`. §11.4.33 binds the two: the closure terminal value MUST agree with the item Type, drawn from this 3-element closed map:

Item `**Type:**`	Closure `**Status:**` value
`Bug`	`Fixed (→ Fixed.md)`
`Feature`	`Implemented (→ Fixed.md)`
`Task`	`Completed (→ Fixed.md)`

The `(→ Fixed.md)` suffix is preserved across all three so the existing migration-discipline tooling (Issues.md → Fixed.md atomic move per §11.4.19) continues to work without per-Type branching. Generators (`generate_issues_summary.sh` , `generate_fixed_summary.sh` , status-counter helpers, the §11.4.23 colorizer) MUST treat the three terminal values as semantically equivalent (all map to "closed, positive evidence captured") while preserving the literal in the emitted document.

No escape hatch. Closing a `Feature` with `Fixed (→ Fixed.md)` or a `Task` with `Implemented (→ Fixed.md)` is a §11.4.33 violation. Pre-build gate (recommended, per consuming project) `CM-CLOSURE- VOCAB-TYPE-AWARE` walks every Fixed.md heading + every Issues.md heading whose `**Status:**` is one of the three terminal values, and asserts the Status value matches the item's `**Type:**` per the table. Paired mutation flips a Bug entry's status from `Fixed (→ Fixed.md)` to `Implemented (→ Fixed.md)` → gate FAILs.

Propagation. Composes with §11.4.15 (item-status tracking), §11.4.16 (item-type tracking), §11.4.19 (Fixed-document column alignment), §11.4.23 (colorisation — the closed-state palette applies regardless of which of the three terminal words is used). [34]

2026 05 15

---

§ . . — Reopened-source attribution mandate (User mandate, - -)

Forensic anchor — direct user mandate (verbatim, 2026-05-15):

> "when we reopen some workable item (bug, task or feature) we MUST HAVE details on who did reopened this and why (- Reopened status needs: by who, AI or User + details in main Issues doc.). For example, reopened by AI or reopened by real User, why - test failed, manual testing detected problem, etc. Adapt our docs for this - Issues, Isssues_SUmmary and Fixed."

Classification: §11.4.17-classified **universal** — every project that uses §11.4.15's `Reopened` lifecycle value benefits from knowing the reopen source + cause. Without this, reopen-thrash patterns (the same item bouncing between Fixed and Reopened across cycles) cannot be diagnosed, and the §11.4.7 demotion-evidence rule loses its forensic counterparty (you cannot evaluate whether a reopen was operator-side observation or agent-side over-restoration without provenance).

The mandate. Every Issues.md (or equivalent project tracker) heading whose `**Status:**` value is `Reopened` MUST carry, within 8 non-blank lines of the heading, a `**Reopened-Details:**` line that captures four sub-facts:

- **By:** `AI` or `User` (the source-of-truth observer who flipped the status). `AI` covers in-loop reopens (test failure, gate regression, captured-evidence retrospect). `User` covers operator- side observations (manual testing, end-user report, design reconsideration).
- **On:** ISO date (`YYYY-MM-DD`).
- **Reason:** one-line cause classification — chosen from a closed vocabulary `{ test-failed | manual-testing-detected | captured-evidence-contradicts | end-user-report | cycle-re-discovered | design-reconsidered }` . Other values are permitted with explicit `Reason: <free text>` annotation but the closed list MUST be tried first.
- **Evidence:** path to or short description of the captured artefact that justifies the reopen — log file, recording, gate failure ID, operator quote, etc. Reopens without evidence are §11.4.6 / §11.4.7 violations: the reopen IS a demotion-from-Fixed classification change, and demotion requires positive evidence captured under the conditions that re-exposed the defect.

The Issues_Summary.md (or equivalent) Status column MUST distinguish the four `Reopened` sub-states by source so a sweep query for "reopens by AI in the last 30 days" is mechanically possible. Suggested column rendering: `Reopened (AI: test-failed)` vs `Reopened (User: manual-testing)` .

No escape hatch. A `Reopened` entry without `**Reopened-Details:**` is a §11.4.34 violation. Pre-build gate (recommended, per consuming project) `CM-ITEM-REOPENED-DETAILS` mirrors `CM-ITEM-OPERATOR-BLOCKED-DETAILS` (Phase 39.AT pattern): walks every actionable heading, scans 8 non-blank lines for a `**Status:** Reopened` line, then continues scanning for `**Reopened-Details:**` . Missing details line emits WARN initially, hardens to FAIL once backlog is fully populated.

Propagation. Composes with §11.4.6 (no-guessing — the Reason must be drawn from the closed vocabulary or explicitly annotated; no `likely/probably/maybe` causes), §11.4.7 (demotion-evidence — reopen IS a demotion from Fixed),

§11.4.15 (item-status tracking — extends the `Reopened` value's discipline), §11.4.21 (Operator- blocked discipline — same audit-line pattern with the same gate shape).

2026 05 19

---

§ . . — ATM-NNN ticket identifier mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-19):

> "Every workable item in Issues.md / Issues_Summary.md / Fixed.md / Fixed_Summary.md MUST carry a stable, unique, auto-incremental ATM-NNN ticket identifier. ATM- prefix, monotonic, never renumbered, append-only — once assigned to an item it stays bound to that item for the lifetime of the project across every reopen, migration, or rename."

Why this anchor exists. §11.4.15 + §11.4.16 + §11.4.19 + §11.4.33 established the lifecycle vocabulary for tracked items, but the items themselves were identified only by §-letter or Fix-# — both of which are **unstable across migrations**. §-letter is heading-local; when an item moves from Issues.md to Fixed.md the §-letter may be reused for a different item. Fix-# is only assigned on closure for items that touched source code; tasks / features / pure-doc items have no stable identifier at all. Cross-references between commits, test logs, captured-evidence artifacts, Reopens.md history (§11.4.55), README.md doc-link rows (§11.4.57), and external systems (Firebase Crashlytics Issue-IDs, Atlassian-style tracker rows) all need an identifier that is **monotonic, stable, and project-wide unique**. §11.4.54 closes that gap.

Operative rule. Every workable item in `docs/Issues.md` AND `docs/Fixed.md` MUST carry a `[ATM-NNN]` ticket identifier in its heading, in the form `## §X.Y. [ATM-NNN] <title>` (or `### §X.Y.` for nested items). NNN is a positive integer, zero-padded to at least 3 digits (`ATM-001` , `ATM-042` , `ATM-127` , …). Identifiers are allocated by a **canonical helper script** (`scripts/testing/`

`assign_atm_ticket_ids.sh`) that maintains an append-only state file (`scripts/testing/.atm_ticket_state.json`). Once assigned to an item, an ATM-NNN MUST NEVER be:

1. **Renumbered** — the bind is permanent.
2. **Reused** for a different item even if the original item is force-deleted (the helper's state file detects collisions).
3. **Decremented** — counter is monotonic-only.
4. **Skipped intentionally** — gaps are forbidden in the assignment sequence (gaps indicate state-file corruption and trigger gate FAIL).

State file format. `scripts/testing/.atm_ticket_state.json` is a JSON-lines file with one record per allocated ID:

```
{"atm_id": "ATM-001", "heading_hash": "<sha256 of normalized heading>",
 "type": "Bug|Feature|Task", "current_location": "Issues|Fixed",
 "current_status": "<lifecycle value>", "reopens_count": <int>,
 "created_at": "<ISO 8601 UTC>", "last_modified": "<ISO 8601 UTC>"}
```

The `heading_hash` is the canonical binding key — when the helper scans Issues.md / Fixed.md, it computes the normalized hash of each heading and looks up an existing ATM-NNN before allocating a new one. This protects against accidental renumbering on heading reflows / wording edits (the hash MUST stay stable; if the heading text changes substantially, the state file's `heading_hash` field is updated in place via an explicit migration command, NOT auto- rebound).

Summary-table column. `docs/Issues_Summary.md` and `docs/Fixed_Summary.md` MUST carry a leftmost `ATM ID` column showing the identifier. Generators (`generate_issues_summary.sh`, `generate_fixed_summary.sh`) MUST emit this column as the first data column so operators / agents can sort + filter on it.

Composition:

- §11.4.15 (Status), §11.4.16 (Type), §11.4.19 (column-alignment Issues↔Fixed), §11.4.33 (type-aware closure terminal values). All four pre-existing mandates compose with §11.4.54 — the ATM-NNN is the new universal join key.

- §11.4.12 + §11.4.53 (Issues_Summary + Fixed_Summary regen on source changes) — the helper MUST be invoked from `sync_issues_docs.sh` so newly added items receive ATM-NNN identifiers before summary regeneration.
- §11.4.55 (Reopens-history per-item docs) — the per-item Reopens.md lives at `docs/issues/<ATM-NNN>/Reopens.md`.
- §11.4.57 (README.md doc-link section) — every Issues / Fixed item may be linked by ATM-NNN in external references.
- §11.4.44 (revision header) — applies to the state file's audit log if one is maintained.

Pre-build gates:

- `CM-ATM-TICKET-IDS-COMPLETE` — every workable-item heading in Issues.md AND Fixed.md MUST carry an `[ATM-NNN]` token matching `\[ATM-[0-9]{3,}\]`. Headings without one FAIL the gate.
- `CM-ATM-TICKET-IDS-UNIQUE` — no two items share an ATM-NNN. The state file's record count MUST equal the union of Issues.md + Fixed.md `[ATM-NNN]` occurrences (no orphan IDs, no duplicates).
- `CM-ATM-TICKET-IDS-MONOTONIC` — the maximum ATM-NNN in the state file MUST equal `record_count` (no gaps).
- `CM-COVENANT-114-54-PROPAGATION` — anchor literal `§11.4.54` present across canonical files (`constitution/Constitution.md`, `constitution/CLAUDE.md`, `constitution/AGENTS.md`, parent consumer's `CLAUDE.md` + `AGENTS.md`).

Paired mutations (per §1.1):

- Remove `[ATM-NNN]` from one Issues.md heading → `CM-ATM-TICKET-IDS-COMPLETE` FAILs.
- Duplicate an ATM-NNN in the state file → `CM-ATM-TICKET-IDS-UNIQUE` FAILs.
- Delete ATM-NNN=2 from a 5-record state file → `CM-ATM-TICKET-IDS-MONOTONIC` FAILs (gap at 2).
- Strip `§11.4.54` literal from `constitution/CLAUDE.md` → `CM-COVENANT-114-54-PROPAGATION` FAILs.

No escape hatch. No `--skip-atm-assignment`, `--renumber`, or `--no-atm-id-required` flag exists. The discipline exists because unstable item identity is the exact failure mode that produces "the test was passing last week, what changed?" sessions where the operator cannot tell whether a given item is the same one tracked before or a coincidentally similarly-named successor.

Classification: universal (per §11.4.17). Applies to every project consuming the constitution submodule that maintains an Issues / Fixed split (cf. §11.4.53 scoping). Projects that do not maintain such a split are NOT_APPLICABLE.

2026 05 19

§ . . — Reopens-history tracking + per-item Reopens.md doc (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-19):

> "Add a Reopens-count column to Issues / Issues_Summary / Fixed / Fixed_Summary. For any item whose reopens-count > 0, create docs/issues/ATM-NNN/Reopens.md (+ HTML + PDF) with comprehensive reopen history + Fixed cycles. Each reopen MUST include By (AI / User), On (date), Reason, Evidence, and each Fixed-marking with its reasoning chain."

Why this anchor exists. §11.4.34 mandated that Reopened status carry a `**Reopened-Details:**` block — but that block lives in the **current** Issues.md heading and is overwritten on the next state change. A complete reopen / fix-cycle audit trail across the lifetime of an item was not previously canonical. §11.4.55 makes the full history first-class: every item that has been reopened at least once gets a dedicated `Reopens.md` companion doc capturing the entire chronological timeline — every reopen, every closure, every reasoning chain — so an operator / agent investigating "why was this reopened five times" has a single, authoritative source to read.

Operative rule. Every workable item with `reopens_count > 0` MUST have a companion document at `docs/issues/ATM-NNN/Reopens.md` (plus its HTML + PDF exports per §11.4.44 + §11.4.53 sync discipline). The document MUST contain:

1. **Revision header** per §11.4.44 (`Revision` integer + `Last modified` ISO 8601 UTC).
2. **Item identification** — `ATM ID` , `Title` , `Type` , `Current Status` , `Current Location` (Issues.md / Fixed.md), link back to the live heading.
3. **Cycle counters** — `Total reopens` , `Total fixed cycles` (items can reopen multiple times, each followed by a re-closure).
4. **Chronological timeline** — one entry per state-change event, each entry capturing:
   - **By:** `AI` or `User` (per §11.4.34 source attribution).
   - **On:** ISO date.
   - **Event:** `Opened` | `Reopened` | `Fixed` | `Implemented` | `Completed` .
   - **Reason:** the closed-vocabulary value from §11.4.34 (`test-failed` / `manual-testing-detected` / `captured-evidence-contradicts` / `end-user-report` / `cycle-re-discovered` / `design-reconsidered`) for reopens; captured-evidence summary for closures.
   - **Evidence:** path to or short description of captured artefact (test log path, recording path, sink-probe report, operator quote).
   - **Outcome:** what the next state transition was.
5. **Reasoning chain for each closure** — for every `Fixed` / `Implemented` / `Completed` event, the reasoning chain that justified marking it closed (root-cause analysis link, captured-evidence under same-conditions per §11.4.7, gate / mutation pair that defends against regression).
6. **Most-recent state-change pointer** — explicit cross-reference to the current Issues.md / Fixed.md heading.

Summary-table column. Issues_Summary.md and Fixed_Summary.md MUST carry a `Reopens` column showing the integer count. When the count > 0 the cell MUST be a hyperlink to `docs/issues/<ATM-NNN>/Reopens.md` . Generators emit the column; the colorizer (§11.4.23) MAY apply a visual cue when reopens > 2 (repeated-reopen signal — investigate before closing again).

Composition:

- §11.4.34 (Reopened-Details on current heading) is the **per-event** capture; §11.4.55 is the **per-item history** aggregation. Both layers MUST be in sync — the state-machine helper updates Reopens.md on every Reopened-Details parse.
- §11.4.54 (ATM-NNN) provides the stable identifier so the per- item doc has a permanent path even after Issues→Fixed migration.
- §11.4.44 (revision header) applies to every Reopens.md.
- §11.4.45 (Status.md per-integration) — the per-item Reopens.md is the item-scoped analogue of the integration-scoped Status.md.
- §11.4.53 (Fixed_Summary parity) — Reopens column appears on BOTH summary tables symmetrically.

Pre-build gates:

- `CM-REOPENS-DOC-EXISTS-WHEN-COUNT-GT-ZERO` — for every state-file record with `reopens_count > 0`, `docs/issues/<ATM-NNN>/ Reopens.md` MUST exist (+ HTML + PDF). Missing doc FAILs.
- `CM-REOPENS-DOC-REVISION-HEADER` — every Reopens.md carries the §11.4.44 revision header.
- `CM-REOPENS-COL-IN-SUMMARIES` — both Issues_Summary.md and Fixed_Summary.md carry a `Reopens` column.
- `CM-COVENANT-114-55-PROPAGATION` — anchor literal `§11.4.55` across canonical files.

Paired mutations (per §1.1):

- Delete `docs/issues/ATM-NNN/Reopens.md` for an item with `reopens_count=2` → `CM-REOPENS-DOC-EXISTS-WHEN-COUNT-GT-ZERO` FAILs.
- Strip revision header from a Reopens.md → `CM-REOPENS-DOC-REVISION-HEADER` FAILs.
- Remove `Reopens` column from Issues_Summary.md table header → `CM-REOPENS-COL-IN-SUMMARIES` FAILs.
- Strip `§11.4.55` literal from `constitution/CLAUDE.md` → `CM-COVENANT-114-55-PROPAGATION` FAILs.

No escape hatch. No `--skip-reopens-doc`, `--collapse-history`, `--reopens-not-applicable` flag. The discipline exists because loss of reopen history is the exact failure mode that produces "why does this keep happening" sessions with no auditable answer.

Classification: universal (per §11.4.17). Applies wherever §11.4.34 applies (every project tracking Reopened status).

2026 05 27

§ . . — Obsolete status + per-item obsolescence audit mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-27):

> "Regarding the Critical Bug No 6 - Albums cover etc. - Now it seems obsolete after latest request for new behavior when audio and video content is being played. If we do not have Obsolete as an option / status for the ticket in its resolving, we should intoroduce this now and mark obsolete tickets with some light gray background or somthing that will indicate that item is no longer valid. Maybe text - the description to be strikethrough styled as well! Once all this is done please review all existing open or resolved workable items (fixed as well) if they are obsolete - not valid any more. ... There MUST NOT be any mistake! No bluff is allowed of any kind!"

The §11.4.15 Status closed-set is extended with a 4th terminal value `Obsolete (→ Fixed.md)` matching the existing 3-element §11.4.33 closure-vocabulary table (`Bug → Fixed`, `Feature → Implemented`, `Task → Completed`). The `Obsolete` value applies regardless of Type when the workable item is **no longer valid** because: (a) the underlying behaviour the item described has been superseded by a later operator mandate / design pivot (canonical example: Bug #6 Albums cover, superseded by §JU + the 2026-05-27 secondary-display video-playback redesign), (b) the affected feature was REMOVED from the product, (c) the item was duplicated and the canonical entry resolved it, (d) the item was filed against a hardware/topology variant we no longer support.

Mandatory obsolescence audit metadata — every `Obsolete (→ Fixed.md)` heading MUST carry, within 8 non-blank lines of the heading, an `**Obsolete-Details:**` line capturing four sub-facts (mirroring §11.4.21 + §11.4.34 audit-line patterns):

- **Since:** ISO date (`YYYY-MM-DD`).
- **Reason:** one-line cause from the closed vocabulary `{ superseded-by-design-change | superseded-by-later-mandate | feature-removed | duplicate-of | unsupported-topology | not-reproducible }` . (`not-reproducible` = a reported defect that does NOT reproduce on the canonical tree / baseline — an environment / isolated-worktree artifact (PATH / shell / missing-checkout), not a real product defect; the triple-check evidence MUST capture the canonical-tree non-reproduction, per §11.4.6 no-guessing + §11.4.7 demotion-evidence.)
- **Superseding-item:** §-letter reference or User-mandate verbatim quote anchor citing the work that obsoleted it.
- **Triple-check evidence:** path to captured evidence (git log, code grep, runtime behaviour) confirming the item is genuinely no longer valid. Per operator mandate "There MUST NOT be any mistake" — bare assertion is forbidden; positive captured evidence per §11.4.6 is mandatory.

Visual treatment — the §11.4.23 colorizer MUST style `Obsolete` cells with `cell-status-obsolete` class: light-gray background `#E0E0E0` + strikethrough text on the row's description cell. Operator mandate verbatim: "mark obsolete tickets with some light gray background or somthing that will indicate that item is no longer valid. Maybe text - the description to be strikethrough styled as well".

Audit cadence — at every release-gate sweep (per §11.4.40), the conductor MUST re-evaluate every non-terminal Issues.md item AND every Fixed.md item for obsolescence per the closed-set Reason vocabulary above. New obsolescence findings produce `Obsolete (→ Fixed.md)` migrations atomic per §11.4.19. Triple-check is non-negotiable: each obsolescence claim cites (i) the superseding mandate, (ii) code-state grep confirming the obsoleted behaviour is gone, (iii) runtime/test evidence confirming no test still exercises the obsoleted path.

Pre-build gates `CM-COVENANT-114-90-PROPAGATION` (anchor literal across canonical files) + `CM-ITEM-OBSOLETE-DETAILS` (every `Obsolete` heading carries `**Obsolete-Details:**` line within 8 lines) + `CM-OBSOLETE-COLORIZER-WIRED`

(§11.4.23 colorizer emits `cell-status-obsolete` class on the right cells + the CSS file defines the class). Paired §1.1 mutations strip the anchor literal / delete an Obsolete-Details line / strip the colorizer class — every mutation FAILs its gate.

Composes with §11.4.15 (item-status closed-set extension), §11.4.16 (Type tracking unchanged — Obsolete is orthogonal to Type), §11.4.19 (Fixed-document column-alignment — Obsolete migrates to Fixed.md just like Fixed/Implemented/Completed), §11.4.21 (Operator-blocked details-line pattern — Obsolete-Details mirrors), §11.4.23 (visual cue + grouping — adds `cell-status-obsolete`), §11.4.33 (closure vocabulary — Obsolete is the 4th terminal value), §11.4.34 (Reopened-source pattern — Obsolete uses analogous audit line), §11.4.40 (release-gate sweep — audit cadence trigger), §11.4.42 (iteration discipline — obsolescence audit is a routine cycle step), §11.4.66 (interactive clarification — surface ambiguous obsolescence to operator), §11.4.71 (pre-push fetch — re-check obsolescence post-fetch).

Canonical authority: this Constitution.md §11.4.90 in the HelixConstitution submodule.

Non-compliance is a release blocker. Marking a still-valid item Obsolete is a §11.4 PASS-bluff at the planning layer. Marking a genuinely-obsolete item with a non-terminal Status is documentation drift. No escape hatch — every obsolescence finding follows the triple-check evidence rule above.

2026 05 27

---

§ . . — Summary-doc clarity mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-27):

> "We see in Summary docs - Issues_Summary some not clear one line descriptions - like for example: 'Composes with'. For each workable item in any variant of documentation - long or short descriptions - we MUST HAVE clearly understandable meaning of particular entry of the document! Re-evaluate all of them, fix the descriptions (especially one liners) and make sure that every team member can clearly understand what that particular or any particular workable item from the list is exactly about! There cannot be misunderstanding or unclearity of any kind and no bluff allowed!"

Every short-form summary entry (Issues_Summary.md, Fixed_Summary.md, README.md doc-link section, Status_Summary.md page 1 + 2, every other one-liner derived from a long-form tracker entry) MUST contain a description that is **self-contained + meaningfully informative** about WHAT the item is, not a fragment of the long-form body. Specifically forbidden one-liner anti-patterns:

- **Section labels** as descriptions: `Composes with`, `Closure criteria`, `Fix direction`, `Forensic anchor`, `Reopened-Details`, `Operator-Block-Details`, `Obsolete-Details`, `Summary`, `Status`, `Type`, `Severity`, etc. — these are section *labels* inside the long-form entry, NEVER appropriate as the one-liner description.
- **Bare numeric / categorical fragments**: `Critical`, `High`, `Medium`, `Low`, `Bug`, `Feature`, `Task`, `Fixed`, standalone — these are metadata column values, not descriptions.
- **Generic restatements of Status**: `In progress`, `Queued`, `Reopened`, `Operator-blocked`, `Fixed`, standalone — again metadata column values.
- **Section-marker echoes**: anything matching `^[A-Z][a-z]+ (with|by|on|in|to|for):?$` patterns where the right-hand side is empty.
- **§-letter alone**: `§KB`, `§EU`, etc. — the section letter is the identifier, not the description.

Required one-liner shape — every summary entry's description column MUST contain a complete clause (≥ 6 words OR ≥ 40 characters, whichever is longer) that names the SUBJECT (component / app / behaviour / defect class) + the PROBLEM or GOAL (what's broken / what's being added / what's being audited). Example transformations:

- Bad: "Composes with"
- Good: "MPV fork comprehensive secondary-display playback + interaction-pattern + chaos test suite (§KB-2)"
- Bad: "Critical"
- Good: "HDMI audio total loss with 'command error 1' on D3 post-reflash (Tier 1/2/3 source-side fixes landed, REQUIRES_REBUILD validation pending)"
- Bad: "§JU"
- Good: "Presenter 2nd-display behaviour simplification + wake-on-play + notification dim toggle (umbrella, REQUIRES_REBUILD pending validation)"

Generator-level enforcement — `generate_issues_summary.sh` + `generate_fixed_summary.sh` + `update_readme_doc_links.sh` + `generate_status_summary.sh` MUST extract the one-liner from the **H1/H2 heading line** of the source long-form entry (which already encodes the subject + context per §11.4.15 + §11.4.16 + §11.4.54 ATM-NNN convention), NEVER from arbitrary downstream text. The generators MUST refuse to emit a row whose description matches any of the forbidden anti-patterns above — emitting `(MISSING DESCRIPTION — fix source heading)` placeholder instead, with the offending row visually highlighted.

Pre-build gate **`CM-SUMMARY-CLARITY-DESCRIPTIONS`** scans every `*_Summary.md` for the forbidden anti-patterns above; any match FAILs the gate. Paired §1.1 mutation injects "Composes with" into a Summary cell → gate FAILs.

Audit cadence — at every release-gate sweep (§11.4.40 + §11.4.42 step 4), re-evaluate every summary row for clarity. Operator-facing exports (HTML + PDF) MUST never ship with an anti-pattern row.

Composes with §11.4.12 (Issues_Summary sync — clarity is a property of the generated row), §11.4.19 (Fixed-document column alignment — clarity applies symmetrically), §11.4.23 (colorizer — anti-pattern rows highlighted), §11.4.44

(revision header — generator updates revision on clarity regeneration), §11.4.53 (Fixed_Summary parity), §11.4.56 (Status_Summary two-audience format — page 1 audience MUST get the clearest descriptions), §11.4.57 (README doc-link rows — same anti-pattern rule), §11.4.59 (README always-sync), §11.4.60 (composite docs always-sync), §11.4.65 (universal MD export), §11.4.74 (mechanical enforcement — pre-commit hook for summary clarity).

Canonical authority: this Constitution.md §11.4.91 in the HelixConstitution submodule.

Non-compliance is a release blocker. A summary row whose description is an anti-pattern fragment is severity-equivalent to a §11.4 PASS-bluff at the documentation layer — it implies the item is documented when it is not. No escape hatch. [92]

2026 05 27

---

§ . . — Multi-pass change-evaluation discipline (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-27):

> "Every change to the project or codebase we do MUST BE evaluated in several passes and in in-depth analisys for potential new issues or problems it can introduce! We MUST BE sure that: change nothing breaks, we do validate and verify this in codebase and through comporehensive deep research, that no new issues or problems of any kind is bringing and that main task is achieved with particular change with no bluff of any kind! After we do change or set of changes this mandatory steps MUST BE taken! Then, and only then, when we have validate and verified codebase and perfromed full analisys and taken into the account all existing knowledge(s) and information sources and confirmed all points we have mentioned now, then this can be accepted as change that is going to be commited, pushed, tested, relased and so on! Write as many as required additional documentation and notes abouth our codebase, the system, or the changes so we can achieve this goals!"

Every non-trivial change to the project codebase MUST pass a multi-pass evaluation BEFORE the change is accepted as commit-ready. The discipline expands §11.4.4 (test-interrupt-on-discovery) + §11.4.8 (deep-web-research) + §11.4.43 (TDD-fix) + §11.4.50 (deterministic consistency) + §11.4.82 (Phase 1 forensic before any speculative patch) into an explicit pre-commit evaluation pipeline.

The 5-pass evaluation contract (mandatory, ALL passes must hold):

Pass 1 — Main-task verification. The change achieves the stated main task. Captured-evidence per §11.4.5 / §11.4.69 demonstrates the user-visible behaviour now works AS INTENDED. NO inferred / NO assumed / NO "should work" — only captured evidence.

Pass 2 — Regression-blast-radius analysis. Every file the change touches AND every file that imports / sources / references the changed file is enumerated. For each, the agent demonstrates (via grep / dependency-graph / actual test run) that the change does not break the existing contract. Examples: function signature changes → audit every caller; HAL config change → audit every test consuming the config; gate addition → audit every paired meta-test mutation. Captured evidence per §11.4.5.

Pass 3 — Cross-feature interaction analysis. Beyond direct-dependency blast radius, the change is evaluated against parallel features that share state, timing, hardware, or shell environment. Examples: audio HAL change → check video routing (Fix #88 SurfaceView Z-order timing); WiFi roaming change → check BT A2DP coex (Fix #70); test-script addition → check §11.4.84 working-tree quiescence + §11.4.89 background-test discipline. Captured evidence per §11.4.5.

Pass 4 — Deep-research validation. Per §11.4.8, the chosen approach is verified against external state-of-the-art (official docs, vendor guides, open-source codebases, kernel mailing list, AOSP gerrit, GitHub issues). Either the change cites an external precedent OR the literal "NO external solution found — original work" is recorded. CodeGraph queries per §11.4.78 + §11.4.79 surface intra-codebase precedents.

Pass 5 — Anti-bluff confirmation. Per §11.4 / §11.4.1 / §11.4.6 / §11.4.27 / §11.4.50 / §11.4.52 / §11.4.69 / §11.4.83 the change is verified to NOT introduce any new bluff surface — no metadata-only PASS, no config-only PASS, no script-bug FAIL-bluff, no "tests pass but feature doesn't work" anti-pattern. Captured evidence directories per §11.4.5 reference the exact passing paths.

Documentation requirement. Each Pass produces written documentation (commit-message footers OR `docs/` entries OR `qa-results/` evidence) demonstrating the pass was completed. A change without all 5 passes documented is severity-equivalent to a §11.4 PASS-bluff at the development-process layer.

Acceptance criteria. Only AFTER all 5 passes complete with documented evidence may the change be: (a) committed via `commit_all.sh`, (b) pushed via §11.4.88 background-push, (c) tested via §11.4.89 background test, (d) released / tagged per §11.4.40.

No escape hatch. No `--skip-multi-pass`, `--single-pass`, `--bluff-permitted`, `--fast-commit` flag exists. The 5 passes apply to every non-trivial change per the operator's verbatim mandate. Trivial changes (typo fixes, doc revision-header bumps, MD-export regeneration) are exempt ONLY when (i) the change touches zero source code AND (ii) the commit message explicitly cites the exemption.

Composes with §11.4 / §11.4.1 (anti-bluff baseline), §11.4.4 (test-interrupt-on-discovery — Pass 2 + Pass 3 trigger this), §11.4.5 (captured-evidence quality — every Pass's evidence MUST be quality-checked), §11.4.6 (no-guessing — Pass 1-3 require captured evidence not hypothesis), §11.4.8 (deep-research — Pass 4 expansion), §11.4.20 / §11.4.70 (subagent-driven — Pass 2-3 dispatchable to subagents per parallel analysis), §11.4.27 (no-fakes-beyond-unit — Pass 5 enforces), §11.4.42 (iteration discipline — multi-pass IS the iteration), §11.4.43 (TDD-fix — RED→Pass-1, GREEN→Pass-2-5), §11.4.50 (deterministic consistency — Pass 1 captured evidence MUST be N=3 reproducible), §11.4.52 (autonomous validation — Pass 5 anti-bluff confirmation gates on autonomous evidence), §11.4.69 (universal sink-side evidence — Pass 1+5 use closed-set taxonomy), §11.4.78 / §11.4.79 (CodeGraph — Pass 3 dependency-graph queries), §11.4.82 (Phase 1 forensic — §11.4.92 generalises Phase 1 to Pass-1+Pass-2+Pass-3 across every change class), §11.4.83 (docs/qa transcript — Pass evidence dir is the QA transcript), §11.4.85 (stress + chaos — Pass 2+3 evidence MUST include

stress + chaos signals when applicable), §11.4.87 (endless-loop — multi-pass runs concurrently across PWUs per §11.4.58), §11.4.89 (background tests — Pass 1-3 long tests run backgrounded).

Pre-build gate `CM-COVENANT-114-92-PROPAGATION` enforces this anchor literal across the canonical fleet. Pre-build gate `CM-MULTI-PASS-EVALUATION-EVIDENCE` audits recent commits for the 5-pass evidence trail (commit message footer OR per-commit `qa-results/<commit-hash>/passes/` directory OR Issues.md / Fixed.md narrative). Paired §1.1 meta-test mutations strip the load-bearing literals → gates FAIL.

Canonical authority: this Constitution.md §11.4.92 in the HelixConstitution submodule.

Non-compliance is a release blocker. A commit landing without the 5-pass evaluation documented is severity-equivalent to a §11.4 PASS-bluff at the development-process layer — it implies the change was vetted when it was not.

2026 05 27

§ . . — SQLite-backed single-source-of-truth for workable items (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-27):

> "There MUST be single source of truth for all of our workable items - SQlite database containing all workable items, all data about it and from there proper scripts (we recommend Go programs) which will update / regennerate all documentation - Issues, Fixed, summary docs, and evrything related! We MUST reduce a chance for sync to be broken between the Db and all documents we have! We MUST BE able to generate always all docs from DB or to re-generate Db from all docs we have in opposite direction if we must! All this MUST BE covered with all supported test types, validated and verified by tests which will produce real proofs and be 100% anti-bluff compatible!"

The current text-based Issues.md / Fixed.md / Issues_Summary.md / Fixed_Summary.md / CONTINUATION.md / Status.md tracker constellation is converted to a **SQLite-database-backed single source of truth** with bidirectional regeneration between DB and Markdown. The DB is the authoritative source; all Markdown / HTML / PDF / Status / Summary surfaces are generator output. Sync drift is mechanically impossible because every regeneration starts from the DB.

Canonical artefacts (all consuming projects MUST adopt):

1. **Database file** at canonical path `docs/workable_items.db` (SQLite 3, **TRACKED in git, NEVER gitignored** per User mandate 2026-05-27 amendment). The DB IS the single source of truth — gitignoring it would defeat the SSoT mandate at the version-control layer. Every commit that mutates workable items MUST stage + push the DB file alongside the Markdown regen output. The DB is an exception to §11.4.30 build-artefact gitignore — it is NOT a build artefact, it IS authoritative source data. SQLite's WAL mode produces a sidecar `*.db-wal` + `*.db-shm` which ARE gitignored (transient WAL files; their state is checkpointed into the main `.db` file by `workable-items sync` or any explicit `PRAGMA wal_checkpoint(TRUNCATE)`).
2. **Schema (mandatory minimum tables):**
   - `items` — primary key `atm_id` per §11.4.54; columns `type` (§11.4.16 closed-set), `status` (§11.4.15 + §11.4.90 closed-set including Obsolete), `severity`, `title`, `description` (≥ 6 words / ≥ 40 chars per §11.4.91), `created_at`, `last_modified`, `current_location` (Issues|Fixed), `forensic_anchor`, `closure_criteria`, `composes_with` (JSON array of refs).
   - `item_history` — append-only audit log: `atm_id`, `event_type` (Opened|Updated|Reopened|Fixed|Implemented|Completed|Obsolete), `by` (AI|User per §11.4.34), `on_date`, `reason` (closed vocabulary per §11.4.34 + §11.4.90), `evidence_path`.
   - `obsolete_details` — `atm_id`, `since`, `reason` (§11.4.90 closed-set), `superseding_item`, `triple_check_evidence` (mandatory per §11.4.90).
   - `operator_block_details` — `atm_id`, `what`, `why_exhausted_alternatives`, `unblock_condition`, `who` per §11.4.21.
   - `firebase_metadata` — per §11.4.47.

- `meta` — schema version, last_sync_direction, last_sync_timestamp, integrity_hash.

3. **Go binary** at canonical path `cmd/workable-items/` (separate Go module, NOT part of AOSP build). Subcommands:
   - `workable-items sync md-to-db` — parse Issues.md + Fixed.md (single source of authority during transition window), upsert into DB. Idempotent.
   - `workable-items sync db-to-md` — regenerate Issues.md + Fixed.md + Issues_Summary.md + Fixed_Summary.md + CONTINUATION.md §3 + every Status.md from DB. Idempotent.
   - `workable-items diff` — print DB vs MD divergence. Used by pre-build gate.
   - `workable-items validate` — schema sanity + §11.4.15/§11.4.16/§11.4.33/§11.4.34/§11.4.54/§11.4.90/§11.4.91 invariants per row. FAIL on any violation.
   - `workable-items add <type> <severity> --title <title> --description <description>` — interactive entry with mandatory fields enforced.
   - `workable-items close <atm-id> --status <fixed|implemented|completed|obsolete> --evidence <path>` — terminal transition with mandatory captured-evidence per §11.4.5/§11.4.90.
4. **Operational integration:**
   - `commit_all.sh` pre-commit hook runs `workable-items diff` and refuses if MD/DB diverge.
   - `sync_issues_docs.sh` invokes `workable-items sync db-to-md` instead of legacy bash generators (`generate_issues_summary.sh` becomes a shim wrapping the Go binary).
   - `pre_build_verification.sh` runs `workable-items validate` as a gate; failure increments ERRORS.
5. **Anti-bluff test coverage** (mandatory per §11.4 + §11.4.85 + §11.4.27):
   - Unit tests for every CRUD operation + every schema invariant.
   - Integration test: full round-trip MD→DB→MD with byte-identical re-emission (modulo cosmetic-whitespace normalisation per closed-set tolerance).

- Stress + chaos per §11.4.85: 1000-row insert / concurrent-write (10 writers) / mid-write SIGKILL / corrupt-DB recovery / disk-full handling.
- Paired §1.1 meta-test mutation: strip a CRUD function → unit test FAILs.
- HelixQA Challenge entry `CME-WORKABLE-ITEMS-001` exercising end-to-end DB→MD→DB→MD round-trip.

Bidirectional regeneration guarantee. The DB MUST be reconstructible from the MD docs (for disaster recovery + audit replay) AND the MD docs MUST be reconstructible from the DB (the primary direction during normal operations). Either direction's regeneration produces semantically equivalent output (closed-set tolerance for cosmetic whitespace + section-ordering noise).

Cross-project propagation. Every consuming project that adopts this constitution submodule inherits the SQLite-SSoT approach. The Go binary lives in the constitution submodule (`constitution/scripts/workable-items/`) so consumers reference it from there per §11.4.74 catalogue-first discipline — never reimplement.

Migration path (per §11.4.42 iteration discipline + §11.4.58 PWU pipeline):

- **Phase 1**: file `§LA` Issues entry tracking the migration, scope-locked.
- **Phase 2**: Go binary scaffold + schema DDL committed.
- **Phase 3**: `sync md-to-db` lands + initial migration captures current state.
- **Phase 4**: `sync db-to-md` lands + byte-identical round-trip CI gate.
- **Phase 5**: existing generators (`generate_issues_summary.sh` etc.) become Go-binary shims.
- **Phase 6**: legacy text-direct edits prohibited (pre-commit hook).

Pre-build gates `CM-COVENANT-114-93-PROPAGATION` (anchor literal in canonical fleet) + `CM-WORKABLE-ITEMS-DB-PRESENT` (DB regen mechanism per §11.4.77 + DB schema-version current) + `CM-WORKABLE-ITEMS-MD-DB-IN-SYNC` (diff returns empty). Paired §1.1 meta-test mutations strip the load-bearing literals → gates FAIL.

Composes with §11.4 (anti-bluff covenant — DB is the captured-evidence registry), §11.4.12 (Issues_Summary sync — DB-driven), §11.4.15 (Status closed-set — DB schema enforces), §11.4.16 (Type closed-set — DB schema enforces), §11.4.17 (universal-vs-project — DB schema is universal), §11.4.19 (Fixed-document column-alignment — DB query emits both Summaries from same query),

§11.4.21 (Operator-blocked details — `operator_block_details` table), §11.4.27 (no-fakes-beyond-unit — DB integration tests exercise real SQLite), §11.4.30 (.gitignore + regeneration mechanism — DB file gitignored with `workable-items sync` as regen), §11.4.33 (closure vocabulary — DB enforces), §11.4.34 (Reopened source attribution — DB `item_history` table), §11.4.42 (iteration discipline — 6-phase migration), §11.4.43 (TDD — RED before each phase), §11.4.44 (revision header — Markdown output preserves), §11.4.45 (Status.md per integration — generator queries DB by domain), §11.4.50 (deterministic consistency — round-trip MD→DB→MD byte-identical), §11.4.51 (LIVE_ADB_FIRST — N/A, host-only), §11.4.52 (autonomous validation — DB integrity is autonomously verifiable), §11.4.53 (Fixed_Summary parity — DB ensures), §11.4.54 (ATM-NNN — DB primary key), §11.4.55 (Reopens-history doc — DB `item_history` queryable), §11.4.56 (Status_Summary parity — DB queries produce both pages), §11.4.57 (README doc-link section — DB drives), §11.4.58 (parallel PWU — Phase 1-6 land as separate PWUs), §11.4.60 (composite always-sync — DB→MD→HTML+PDF), §11.4.65 (universal MD export — final stage), §11.4.74 (catalogue-first — Go binary in constitution), §11.4.83 (docs/qa transcript — DB queries serve as audit replay), §11.4.85 (stress + chaos — DB integration test scope), §11.4.86 (roster/corpus auto-sync — DB schema accommodates roster tables), §11.4.87 (endless-loop — DB queries are zero-idle-friendly), §11.4.89 (background tests — DB validation runs detached), §11.4.90 (Obsolete status — `obsolete_details` table), §11.4.91 (Summary clarity — DB enforces description-floor at insert time), §11.4.92 (multi-pass evaluation — every DB schema migration follows 5-pass).

Canonical authority: this Constitution.md §11.4.93 in the HelixConstitution submodule.

Non-compliance is a release blocker. A consuming project that maintains text-based trackers without the DB-SSoT migration plan filed + Phase progression actively in flight is severity-equivalent to a §11.4 PASS-bluff at the data-architecture layer — the sync-drift surface §11.4.93 closes IS a §11.4 violation vector.

---

§ . . — Workable-items SQLite DB is TRACKED in git, NEVER gitignored (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-27):

> "We shall not Git ignore our workable items SQlite DB since it is our single source of truth for current working items we have and all data related to it! We MUST fix this if DB is Git ignored! ... workable items SQlite DB regularly commited and pushed to all upstreams!"

The §11.4.93 workable-items SQLite database at `docs/workable_items.db` is the **AUTHORITATIVE source of truth** for every workable item in every consuming project. It MUST be:

- **TRACKED in git** at canonical path `docs/workable_items.db`. NEVER gitignored regardless of file-size or "build-artefact-class" heuristics — the DB is NOT a build artefact, it IS authoritative source data.
- **Committed alongside every workable-item state change.** Every `workable-items sync md-to-db` invocation that mutates DB state MUST stage + commit + push the DB file in the same commit as the corresponding MD changes per §11.4.19 atomic-move discipline.
- **Pushed to every upstream per §2.1** — every mirror of every consuming project carries the latest DB so a fresh clone from any mirror reconstructs the full workable-items inventory without consulting any other source.
- **WAL-checkpointed before commit** — SQLite's WAL mode produces transient `*.db-wal` + `*.db-shm` sidecars. Those ARE gitignored (per §11.4.30 transient/cache classification). The `workable-items` Go binary MUST execute `PRAGMA wal_checkpoint(TRUNCATE)` before any commit-stage step so the main `.db` file carries the authoritative state and the sidecars are safely discardable.
- **NEVER force-rewritten** without §9.2 hardlinked-backup + operator authorization. The DB's commit history IS the workable-items audit trail; rewriting it is data-loss equivalent to force-pushing a tracked Issues.md history.

§11.4.30 carve-out. This anchor is an explicit named exception to §11.4.30's build-artefact / data-file gitignore rule. The DB file at `docs/workable_items.db` is **NOT** a build artefact and is **NOT** transient data — it is the project's SSoT registry whose value increases monotonically as work lands. The §11.4.77 regeneration mechanism applies to other gitignored bulk data (`.git-backup-*` , `RKTools/linux/` , etc.) — NOT to this DB.

§11.4.93 amendment. §11.4.93's earlier text describing the DB as "gitignored per §11.4.30 with §11.4.77 regeneration mechanism" is hereby **AMENDED**: the DB is TRACKED, not gitignored. The Markdown trackers (Issues.md / Fixed.md / Summaries / Status.md fleet / CONTINUATION.md) remain tracked AND derived; the DB is tracked AND authoritative. Both directions of regeneration (`workable-items sync db-to-md` + `workable-items sync md-to-db`) continue to land per §11.4.93's bidirectional round-trip guarantee.

Pre-build gates `CM-COVENANT-114-95-PROPAGATION` (anchor literal across canonical fleet) + `CM-WORKABLE-ITEMS-DB-TRACKED` (`git ls-files docs/workable_items.db` returns non-empty if DB exists on disk + `.gitignore` does NOT exclude it). Paired §1.1 meta-test mutation adds the DB to `.gitignore` → gate FAILs.

Composes with §11.4.93 (SQLite-SSoT — §11.4.95 amends its tracking-policy clause), §11.4.30 (.gitignore discipline — explicit carve-out anchored here), §11.4.77 (regeneration mechanism — does NOT apply to the DB; the DB IS the source), §2.1 (multi-upstream push — DB pushed to every mirror), §9.2 (data safety — destructive DB ops require hardlinked-backup + operator authorization), §11.4.41 (force-push merge-first — applies if DB history ever needs rewrite).

Canonical authority: this Constitution.md §11.4.95 in the HelixConstitution submodule.

Non-compliance is a release blocker. A consuming project that gitignores its workable-items DB is severity-equivalent to a §11.4 PASS-bluff at the source-of-truth layer — it implies SSoT exists when it is in fact ephemeral local-only state.

---

§ . . — Participant identity, attribution & notification-tagging (User mandate, - -)

Short tag: `participant-attribution-tagging`.

Forensic anchor — verbatim user mandate (2026-05-31):

> "Every supported messenger must relate messages to participants (Subscribers/Users); the same logical person may have a different username on every messenger. Workable items must carry who created them and who they are assigned to. Notifications must @-tag the right participant — but never the operator (who drives the system) and never the system agent."

This anchor binds participant identity, workable-item attribution, and notification @-tagging as MANDATORY constraints on every consumer that ships a messenger/notification surface (Herald and its flavor binaries are the reference implementation; a consuming project and any future messenger-bearing project inherit per §11.4.35). The detailed normative spec is the Herald design contract `docs/design/PARTICIPANT_ATTRIBUTION.md` (model, columns, attribution rules, tagging matrix) — this section restates its load-bearing constraints; implementations code against the contract, not against a paraphrase.

(A) Participant identity — logical person + per-channel handle. Every supported messenger MUST relate inbound + outbound messages to a **Participant** (a logical Subscriber/User). A single logical person/agent MAY carry a DIFFERENT username on every messenger; identity is therefore modelled as a logical subscriber (canonical, messenger-neutral `handle`, `kind ∈ {human, agent, service}`) PLUS per-channel aliases (`channel`, `channel_user_id`, the per-channel `@username` used for tagging). The **canonical handle** is the string stored in attribution columns — the closed set is `Claude` (the reserved system-agent sentinel; `kind=agent`) OR a human's canonical handle (a subscriber `@username`, messenger-neutral, resolved per-channel via the alias table).

(B) Operator designation — env var, not a DB flag. The **Operator** is the one human who drives the system via the agent CLI. The Operator is designated by the environment variable `HERALD_<CHANNEL>_OPERATOR_USERNAME` (e.g.

`HERALD_TGRAM_OPERATOR_USERNAME`, `HERALD_SLACK_OPERATOR_USERNAME`) — per messenger, NOT a database flag. The Operator is a normal Participant whose canonical handle equals that env value.

(C) Workable-item attribution — `created_by` + `assigned_to`. Every workable item MUST carry two canonical-handle columns: `created_by` and `assigned_to`. Attribution rules: opened via the agent CLI prompt (operator-driven) → `created_by = Operator`; opened by the system/agent detecting an issue/task/improvement/missing-feature → `created_by = "Claude"`; received THROUGH the system as a subscriber message → `created_by =` the sender's resolved canonical `@username`. `assigned_to` **defaults to the Operator** and MAY be overridden explicitly (a prompt/message assigning to `@someoneelse`). Both columns store the canonical handle string and are self-contained in the SSoT + its Markdown export. Legacy items predating this anchor carry empty (`""`) attribution and MUST still parse + validate.

(D) Notification @-tagging matrix. On any workable-item event, the outbound notification dispatched to each messenger group @-tags the participant(s) who must be aware, resolved to that channel's `@username`:

```
mentions = {}
if assigned_to is a human handle AND assigned_to !=
Operator:                    mentions += assigned_to
if created_by  is a human handle AND created_by != Operator AND
created_by != "Claude":  mentions += created_by
# "Claude" is NEVER tagged (it is the system agent).
# the Operator is NEVER tagged (drives the system; no self-ping).
# de-dup; for each mention resolve the @username on the target channel —
skip if the participant has no alias there.
```

This satisfies the operator's stated cases exactly: assigned-to-Operator → no tag; opened-by-Operator-assigned-to-another → tag the assignee; opened-by-a-non-Operator-non-Claude-subscriber → tag the creator.

(E) Anti-bluff (MANDATORY, composes with §11.4 / the end-user quality covenant). Every layer ships unit + integration + E2E + full-automation tests producing REAL captured evidence (no metadata-only / absence-of-error PASS, no false positive/negative): real SQLite round-trip with the new columns (byte-identical, including legacy fixtures that DON'T carry the fields); the tagging matrix proven by a truth-table test with a mutation that flips one cell forcing a FAIL; an E2E real-item-

event → real-dispatched-message asserting the exact `@username`s AND a NEGATIVE case proving the Operator is NOT tagged. Evidence committed under `docs/qa/<run-id>/`.

Composes with §11.4 + §11.4.1..§11.4.16 (end-user quality / anti-bluff covenant — sub-rule (E) is bound by it), §11.4.5 / §11.4.69 (captured evidence), §11.4.50 (deterministic consistency), §11.4.91 (workable-item field clarity), §11.4.93 / §11.4.95 (the SQLite workable-items SSoT the `created_by` / `assigned_to` columns extend), §1.1 (paired-mutation proof of the tagging matrix).

Classification: universal (§11.4.17) — participant identity, item attribution, and notification @-tagging is a reusable discipline for ANY project that ships a messenger/notification surface with a workable-items registry; the env-var operator designation + canonical-handle model + tagging matrix are vendor-neutral. Projects with NO messenger surface inherit the anchor latently (it binds the moment they ship one) — the §11.4.96 "principle binds even absent the surface" restatement pattern.

4-layer coverage per §11.4.4(b). Propagation gate `CM-COVENANT-114-104-PROPAGATION` enforces the literal anchor `11.4.104` across the canonical consumer fleet (parent + owned-submodule CLAUDE.md / AGENTS.md / QWEN.md). Paired §1.1 meta-test mutation strips the `11.4.104` literal from a consumer file → the gate FAILs. (Gate-code implementation lands as a separate work item; this anchor defines the contract.)

Canonical authority: this Constitution.md §11.4.104 in the HelixConstitution submodule; detailed spec Herald `docs/design/PARTICIPANT_ATTRIBUTION.md`. All consuming projects restate + cite via §11.4.35 inheritance.

Non-compliance is a release blocker. No escape hatch — no `--skip-attribution`, `--no-participant-tagging`, `--tag-operator-anyway`, `--attribution-later` flag exists.

2026-06-03

§. . — Structural-impossibility won't-fix classification mandate (research-derived, - -)

Short tag: `structural-impossibility-wont-fix`.

Forensic anchor (genericised, 2026-06-03). Deep research (§11.4.8) into relocating protected (content-protection / secure-surface) video to a secondary display proved — from authoritative platform/HDCP documentation plus reproducible captured behaviour (a secure surface is blanked on any output lacking the secure flag; mirror/screencap of a secure layer returns black) — that the goal is **structurally impossible by platform design**, not a missing feature or an unsolved engineering problem. Without a durable classification, such a goal is re-investigated every cycle: each new agent / operator re-reads the same sources, re-runs the same probes, re-derives the same impossibility, and burns the same effort — an anti-pattern that compounds across cycles.

The mandate. When deep research per §11.4.8 PROVES (with cited authoritative sources AND, where applicable, reproducible captured evidence) that a goal is structurally impossible on the target platform — i.e. forbidden by the platform's design / a hardware/protocol constraint / a documented kernel-or-API limitation, not merely unimplemented or hard — the goal MUST be: (1) **classified** `Won't-fix` **and closed** per the §11.4.90 terminal-status discipline, with obsolescence/closure reason `structurally-impossible` (a value in the §11.4.90 closed reason vocabulary); (2) **documented with the impossibility evidence** — the cited authoritative source URLs (per §11.4.99 latest-source verification where the platform docs may evolve) PLUS the reproducible probe/captured evidence that demonstrates the constraint, recorded in the tracker entry and the relevant `docs/` guide so the verdict is auditable; (3) **NOT re-attempted in future cycles** — the closed entry is the canonical answer; a future agent that re-opens it MUST cite NEW evidence that the platform constraint changed (per §11.4.34 reopened-source attribution + §11.4.7 demotion-evidence), never merely re-derive the same impossibility; (4) **paired with the correct posture** — the entry MUST state what the project DOES instead (the supported-within-the-constraint behaviour), so "impossible" is never confused with "broken / unhandled".

Honest boundary (§11.4.6). `Won't-fix: structurally-impossible` is reserved for *proven* platform/hardware/protocol impossibility. "We could not find a way" / "it is very hard" / "no time" are NOT structural impossibility — they are `Operator-blocked` (§11.4.21) or open work, and mislabelling them won't-fix to avoid the work is a §11.4 bluff at the planning layer. A future platform change CAN make the impossible possible; the classification is durable but not eternal, and reopening on NEW cited evidence is correct.

Classification: universal (§11.4.17) — proving-then-durably-classifying structural impossibility is a platform-neutral effort-conservation + honesty discipline reusable by ANY project; the consuming project supplies the specific impossible goal, its platform constraint, and the cited evidence per §11.4.35.

Composes with §11.4.6 (no-guessing — the impossibility is stated as FACT with cited evidence, never as "probably can't"), §11.4.7 (demotion-evidence — reopening a won't-fix requires NEW positive evidence the constraint changed), §11.4.8 (deep-web-research — the proof is the research output; "NO external solution found — structurally impossible" is the §11.4.8 citation), §11.4.34 (reopened-source attribution — a reopen of a structurally-impossible item attributes who reopened + the new evidence), §11.4.90 (Obsolete/terminal-status — `structurally-impossible` is a closure reason in the §11.4.90 closed vocabulary; the colorizer marks it like any closed item), §11.4.99 (latest-source verification — the cited platform docs MUST be the latest where they can evolve, so the impossibility verdict is not stale).

Propagation. Propagation gate `CM-COVENANT-114-112-PROPAGATION` enforces the literal anchor `11.4.112` across the consumer fleet; paired §1.1 meta-test mutation strips the literal → the gate FAILs. Recommended per-family gate `CM-WONT-FIX-STRUCTURAL-IMPOSSIBILITY` (every tracker entry with closure reason `structurally-impossible` carries cited authoritative-source evidence + a stated correct-posture line); paired §1.1 mutation strips the evidence citation from such an entry → the gate FAILs. (Gate-code implementation lands as a separate work item; this anchor defines the contract.)

(5) SCOPE-STATEMENT + ADJACENT-GOAL ENUMERATION — an impossibility verdict is bounded, and MUST say where its edge is (extension, research-derived, 2026-07-17). Forensic FACT (genericised, from a consuming project): a verdict that was TRUE on its own terms — "relocating another application's protected video output to a secondary display **while that application's UI stays in place, without the application's cooperation** is structurally impossible" — LEAKED to an ADJACENT, VIABLE goal it never covered: moving the whole task to the other display. For six weeks the project believed "no platform mechanism exists", while the platform had shipped a public, permission-free SDK call for the adjacent goal several major versions earlier, and the platform vendor's own connected-display documentation PRESCRIBED exactly that call. The impossibility was never re-litigated (clause (3) worked); it was CITED,

unexamined, against a neighbour — and clauses (1)-(4) contain nothing that bounds a verdict's reach. This anchor is ASYMMETRIC without this clause: clause (3) hardens a verdict against reopening ("the closed entry is the canonical answer") while nothing constrains how far it may be applied, so the only mechanical pressure runs in the direction that AMPLIFIES a scope leak. Therefore every `structurally-impossible` verdict MUST additionally: **(a) STATE ITS EXACT SCOPE** — the precise goal/variant the cited evidence actually proves impossible, written as the narrowest claim the evidence supports (every qualifier that is load-bearing — "without the application's cooperation", "while the UI stays put", "on this platform version" — is part of the verdict, not commentary; a qualifier dropped is a verdict widened); **(b) ENUMERATE THE ADJACENT GOALS IT DOES NOT COVER** — the neighbouring variants a reader could mistake for the same goal, each marked NOT-COVERED-BY-THIS-VERDICT (a neighbour whose viability is unknown is recorded as UNKNOWN with a tracked follow-up per §11.4.6 — an unexamined neighbour is NEVER silently absorbed into the impossibility); **(c) BE CITATION-FENCED** — the verdict MUST NOT be cited as the answer for ANY goal outside its stated scope, in planning, investigation, design, or status, without its OWN §11.4.150 deep-research pass and its OWN proof (citing it beyond its fence is asserting an unproven claim as fact — a §11.4.6 violation and a §11.4/§11.4.1 bluff at the verdict layer, of the same class §11.4.199 names when a reproduction that never reaches the precondition is used to conclude "the mechanism never engages"). **Honest boundary (§11.4.6):** an adjacent goal being outside the fence does NOT make it viable — it makes it UNDECIDED, and it earns its own verdict by its own evidence, never by inheritance in either direction. **Why this is a clause and not another narrative:** §11.4.150(A)/(E) already documented this exact failure class in prose (its case study is an impossibility verdict OVERTURNED by the research pass it never had) and REQUIRES the deep-research pass before a `structurally-impossible` verdict is EARNED — yet the leak still happened, because §11.4.150 fires at the seam where a verdict is MINTED for a tracked item, and this leak happened where a verdict was CITED as a premise, which crosses no closure seam at all. Clauses (2) and (4) are already content requirements on the verdict artifact; (5) is the third, and it is the one that bounds blast radius.

Gate contract for clause (5) (extension, 2026-07-17).
`CM-WONT-FIX-STRUCTURAL-IMPOSSIBILITY` is EXTENDED: every `structurally-impossible` entry MUST additionally carry (i) an explicit SCOPE statement and (ii) an adjacent-goals-NOT-covered enumeration (each neighbour

marked not-covered or UNKNOWN + tracked); and (iii) a citation of the verdict against a goal outside its stated scope, without that goal's own §11.4.150 research + proof, → FAIL. Paired §1.1 mutations: strip the scope statement, or the adjacent-goal enumeration, from an impossibility entry → the gate FAILs; cite an in-scope verdict against an enumerated not-covered neighbour → the gate FAILs. Gate-code = a separate work item; this is the contract, not a claim the code has shipped (§11.4.6).

Canonical authority: this Constitution.md §11.4.112 in the HelixConstitution submodule. All consuming projects restate + cite via §11.4.35 inheritance.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--wont-fix-without-proof`, `--reattempt-closed-impossible`, `--skip-impossibility-evidence`, `--impossible-equals-broken`, `--unscoped-impossibility-verdict`, `--skip-adjacent-goal-enumeration`, `--cite-verdict-beyond-scope`, `--neighbour-inherits-impossibility` flag exists.

---

§ . . 2026 06 10 — Workable-item integrity (status+type+id) + comprehensive structured description + bidirectional external-tracker sync + BLOCKED unblock-choices mandate (User mandate, - -)

Forensic anchor — direct operator mandate (2026-06-10): every workable item MUST carry a VALID status, a VALID type, and a stable unique identifier at all times (never one without all three); every item MUST carry a comprehensive, structured, self-explanatory description (what it is, how it manifests, how to reproduce, acceptance criteria); a `BLOCKED` item MUST state WHY it is blocked AND the choices that would unblock it; the single source of truth (the SQLite workable-items DB), the rendered docs, and the external tracker (e.g. ClickUp) MUST be regularly, never-missed, fully synchronized in BOTH directions through the mechanical docs-sync engine; the external-tracker push MUST carry statuses, types, assignee, and sub-tasks, default an unset assignee from an env var, and be idempotent.

This anchor BINDS + STRENGTHENS the existing workable-item discipline (§11.4.15 status / §11.4.16 type / §11.4.54 ATM-NNN id / §11.4.21 Operator-blocked / §11.4.91 description-clarity / §11.4.93 + §11.4.95 SQLite SSoT /

§11.4.106 docs_chain) into ONE integrity contract spanning the three surfaces — DB ↔ docs ↔ external tracker — and adds the operator's three new emphases (D1 the no-item-without-all-three invariant, D2 the comprehensive structured description, D3 the BLOCKED unblock-CHOICES tightening), citing rather than re-authoring its components. The mandate (ALL hold):

(D1) No item without a valid status + valid type + stable id — across ALL three surfaces. Every workable item, at every moment, MUST carry (i) a `**Status:**` from the §11.4.15 / §11.4.21 / §11.4.90 closed set, (ii) a `**Type:**` from the §11.4.16 closed set `{Bug | Feature | Task}`, and (iii) a stable, unique, monotonic, append-only `[<ID-NNN>]` identifier per §11.4.54. An item missing ANY of the three in the DB, in the rendered docs, OR in the external tracker is a §11.4.148 violation — the validator FAILs (non-zero exit, release-blocker). The id is the binding key that keeps the same item identifiable across DB rows, doc headings, and tracker tasks; an id present on one surface but absent on another is a sync-integrity failure.

(D2) Comprehensive structured description per item. Every item MUST carry a comprehensive, self-contained, structured description — at minimum WHAT it is (the §11.4.91 ≥6-word / ≥40-char clear meaning), HOW it manifests (observed user-visible / system behaviour), HOW to reproduce (deterministic steps where applicable, §11.4.115 / §11.4.146 reproduce-first), and ACCEPTANCE CRITERIA (the captured-evidence verdict that closes it per §11.4.123 / §11.4.69). The description lives in the §11.4.93 DB `description` column AND renders into the docs AND pushes to the external tracker — never a one-liner stub, never a §11.4.91 anti-pattern fragment. A "fix-it-later"/placeholder description is a §11.4 PASS-bluff at the planning layer (the operator / a future agent / a teammate cannot understand the item).

(D3) BLOCKED items carry WHY + the unblock CHOICES. Tightens §11.4.21: every item whose status is `Operator-blocked` (incl. any `Blocked` / `BLOCKED` input alias normalised to the canonical `Operator-blocked` value — documented alias, never a silent fork, §11.4.6) MUST carry, beyond the §11.4.21 `Operator-Block-Details` WHAT/WHY/WHO, an enumerated set of UNBLOCK CHOICES — the closed list of decisions/actions that would unblock it (e.g. `[A] grant access X · [B] approve removal of Y · [C] provide credential Z`, mirroring the §11.4.66 2–4-option interactive-clarification shape). The validator FAILs a

BLOCKED item whose unblock field is empty OR carries no enumerated choice marker — "blocked, no way forward stated" is forbidden, the agent / operator must always know the choices that clear the block.

(D4) Bidirectional DB ↔ docs ↔ external-tracker sync, never missed, docs_chain-bound. The §11.4.93 / §11.4.95 git-tracked SQLite DB is the SINGLE SOURCE OF TRUTH; the rendered docs (Issues / Issues_Summary / Fixed / Fixed_Summary / per-item docs + their §11.4.65 / §11.4.106 exports) and the external tracker are its DERIVED surfaces. A full synchronization pass MUST run regularly and on every state change so no surface drifts: DB→docs (`db-to-md`), docs→DB (`md-to-db` , both byte-identical round-trip per §11.4.93), DB→external-tracker push, with a drift-proof §11.4.86 fingerprint (sha256 of the sorted item keyset, NOT mtime) gating freshness so a forgotten sync FAILs a pre-build gate. The sync is bound into the §11.4.106 docs_chain engine as a registered consumer context (never an ad-hoc parallel script) so drift is mechanically caught, never vigilance-dependent.

(D5) Generic external-tracker sync — statuses/types/assignee/sub-tasks, default-assignee env var, idempotent. The external-tracker push MUST carry: each item's status (collapsed onto the tracker's native status set per §11.4.33 when the tracker's status vocabulary is fixed — the API-cannot-create-statuses case is a §11.4.112 structural constraint, the precise value preserved in a description header, never lost), its type, its assignee (a §11.4.104 participant handle — `created_by` / `assigned_to`), and its sub-tasks. An UNSET assignee defaults from a project-supplied env var (the §11.4.104 operator-username pattern), never hardcoded, never logged (§11.4.10). The push MUST be IDEMPOTENT — a dry-run-then-real, match-by-stable-key (`[<ID>]` name prefix / id custom-field) plan, present⇒UPDATE / absent⇒CREATE, rate-limited, credential-redacted, with a captured sink-side `created=N updated=M failed=0` proof per §11.4.69. The tracker-sync MACHINERY stays project-agnostic (§11.4.28) — the consumer registers its concrete tracker (service, list/board id, field map) at runtime via the public API; the constitution names the discipline, not the vendor.

Anti-bluff (§11.4): every sync run + every validator pass carries captured evidence per §11.4.5 / §11.4.69 (the round-trip diff, the fingerprint match, the tracker `created/updated/failed` line under `qa-results/`); a sync claimed-run with no growing evidence trail, a `complete` / `Fixed` item with no acceptance-criteria description, or a BLOCKED item with no unblock choices are all PASS-bluffs at the

workable-item-integrity layer. Honest boundary (§11.4.6): the integrity contract guarantees every item is well-formed + the three surfaces agree, NOT that the item's underlying work is correct — that still crosses §11.4.108 / §11.4.40 / §11.4.123.

Classification: universal (§11.4.17) — the consuming project supplies its concrete DB path, id prefix, external-tracker service + list/board id + field map + default-assignee env var, and docs_chain context per §11.4.35. Composes §11.4.15 / §11.4.16 / §11.4.21 / §11.4.33 / §11.4.34 / §11.4.54 / §11.4.66 / §11.4.86 / §11.4.91 / §11.4.93 / §11.4.95 / §11.4.104 / §11.4.106 / §11.4.112 / §11.4.123 / §11.4.10 / §11.4.28 / §11.4.69 / §11.4.6 / §1.1. Propagation gate `CM-COVENANT-114-148-PROPAGATION` (literal `11.4.148`) + recommended gates `CM-ITEM-INTEGRITY-STATUS-TYPE-ID` (every item carries valid status+type+id on every surface) / `CM-ITEM-COMPREHENSIVE-DESCRIPTION` (every item's description has the WHAT/manifest/repro/acceptance structure) / `CM-BLOCKED-UNBLOCK-CHOICES` (every BLOCKED item enumerates unblock choices) / `CM-TRACKER-SYNC-IDEMPOTENT` (the external-tracker push is bidirectional-fingerprinted + idempotent + docs_chain-bound) + paired §1.1 meta-test mutations (strip the literal → propagation gate FAILs; drop an item's type / id / description-structure / unblock-choices, OR break the tracker-sync idempotency key → the respective gate FAILs; gate-code = separate work item).

EXTENSION — SLICING: an item too big to test is an item whose "done" cannot be proven (2026-08-20). §11.4.148 governs an item's INTEGRITY once it exists (status + type + stable id + comprehensive structured description) and §11.4.239(B) governs its acceptance criteria as Given/When/Then scenarios. Neither governs the item's SIZE AND SHAPE, and the AI-curriculum corpus (module 27) identifies that as the upstream cause of the reopen the other two anchors then have to catch. **(a) THE SLICING CHECKLIST.** Corpus: *"INVEST (Bill Wake) is a checklist for a well-formed item: Independent, Negotiable, Valuable, Estimable, Small, Testable."* Citation discipline (§11.4.6): the corpus expands the acronym but defines ONLY two of its letters, and says so — *"For governance the two load-bearing letters are Small and Testable, and they are linked: an item too big to test end-to-end is an item whose 'done' you cannot prove, so it closes on partial evidence and reopens on the untested part."* This extension therefore mandates the two DEFINED letters and cites the acronym for the remaining four without inventing per-letter obligations the corpus does not state. (The corpus names "SMART Tasks" exactly once, inside a source title, and never expands or mandates it; no SMART

obligation may be grounded on this corpus.) **(b) VERTICAL SLICES, NEVER HORIZONTAL.** The named anti-pattern, verbatim: *"the horizontal slice ('do all the database work,' 'do all the UI') which is never independently valuable or testable — you can't prove it works because it doesn't do anything a user can observe until every slice lands. Vertical slices (a thin end-to-end capability) are testable in isolation, so each closes cleanly."* The worked example: *"'reconcile donations across nine gateways' is not a workable item — it is untestable as one unit. Sliced by INVEST it becomes 'ingest and normalize gateway A's settlement file,' 'detect a provider-only ghost charge,' 'classify a fee/FX residual,' each with its own Given/When/Then and its own evidence."* **(c) WHY IT BINDS HARDER UNDER AGENTS.** Corpus: *"a small, testable item is one a subagent can be dispatched on and verified against a clear oracle; a big fuzzy item is one an agent will report 'done' on with nothing to check it against."* This is the §11.4.240 / §11.4.262 problem entering at the item-definition layer: an item with no clear oracle cannot have its completion verified by anyone, so the producer's self-report becomes the only available signal. **(d) READY-BEFORE-DISPATCH.** The corpus names two practices that pin the criteria before code — **three amigos** (*"the business/product view … the development view … the testing/QA view. The testing voice is the one that surfaces the missing failure scenarios before they become reopens. The output is a shared, written understanding — not a hallway agreement"*) and **specification by example** (*"pin the requirement with concrete examples that become the acceptance tests: real inputs and expected outputs, including the awkward ones"*). **The mandate:** an item MUST be sliced small enough that its own acceptance criteria are provable end-to-end from its own captured evidence, and MUST be a vertical slice — a thin end-to-end capability a user can observe — never a horizontal layer. An item that cannot be closed on its own evidence is not ready for dispatch and MUST be re-sliced before work begins. Honest boundary (§11.4.6): slicing makes "done" PROVABLE; it does not make the work correct (that remains §11.4.108 / §11.4.115 / §11.4.185), and "small" is a judgement about testability-in-isolation, NOT a story-point number, an hour count, or a line-count — the corpus states no size metric and this extension mandates none. Recommended mechanism gate `CM-ITEM-SLICED-SMALL-AND-TESTABLE` (every item carries acceptance criteria whose satisfaction is decidable from that item's own evidence, and is a vertical slice; a horizontal-layer item, or one whose criteria depend on a sibling item landing, → FAIL naming the item) + paired §1.1 mutation (re-shape one item into a horizontal layer whose criteria cannot be met alone → the gate MUST

FAIL; golden-FALSE per §11.4.201(1): a legitimately-dependent item that is nonetheless independently testable once its dependency is met MUST NOT fire it). Gate-code = separate work item, NOT claimed shipped (§11.4.6 / §11.4.227).

Canonical authority: constitution submodule `Constitution.md` §11.4.148. Non-compliance is a release blocker. No escape hatch — no `--item-without-status`, `--item-without-type`, `--item-without-id`, `--stub-description-OK`, `--blocked-without-choices`, `--skip-tracker-sync`, `--one-way-sync-OK`, `--non-idempotent-tracker-push`, `--hardcode-assignee` flag.

§ . . — Per-workable-item testing-diary mandate (User mandate, - -)

Forensic anchor — direct operator mandate (2026-06-10): every workable item MUST carry a TESTING DIARY — a chronological, append-only record of every test run against it, each entry capturing date/time, who tested (tested-by), the result, in-depth observations (the technical background a future fix needs), and the action taken + why (whether the run changed the item's status, and the reasoning); both an in-depth diary AND an at-a-glance summary; modelled on the external tracker as a sub-task of the item with a `{TODO | In-progress | Completed}` lifecycle; persisted in the SQLite single source of truth + exported to all required formats + pushed to the external tracker; bound into the mechanical docs-sync engine so it is never missed; minimal-LLM (deterministic tooling, the prose authored by whoever ran the test); under 100% test-coverage discipline.

The testing diary is the evidentiary HISTORY layer for a workable item — distinct from the §11.4.93 `item_history` (which records lifecycle STATE transitions) and from the §11.4.55 Reopens.md (which aggregates reopen cycles): the diary records TEST EXECUTIONS, each of which may or may not cause a state transition. The mandate (ALL hold):

(a) Append-only per-item diary table in the SQLite SSoT. A `test_diary` table (additive to the §11.4.93 / §11.4.95 schema, never a competing source) records one row per test run, soft-keyed to the item's stable id, with at minimum: ISO-8601-UTC `date_time` (sortable, deterministic §11.4.50); `tested_by` from a closed set (e.g. `User | Operator | AI-agent | HelixQA`); `result` from the §11.4.45 closed verdict vocabulary `{PASS | FAIL | SKIP}` + a short detail; in-depth

`observations` (long markdown — facts or explicit `UNCONFIRMED:` / `PENDING_FORENSICS:` per §11.4.6, never "likely"; this is the background a future fix relies on); `action_taken` + a `status_changed` flag + from/to (did this run change the item's status, and WHY/why-not); and a §11.4.69 `evidence_path` + `feature_class` — with a structural constraint that a `PASS` row WITHOUT a non-empty evidence path is impossible (a PASS-bluff is rejected by the schema itself).

(b) In-depth diary + at-a-glance summary. Per item, BOTH an in-depth diary document (one section per run, full observations + action + evidence link) AND a derived at-a-glance summary (a rollup VIEW — total/pass/fail/skip runs, last verdict, last run, status-change count, distinct testers, distinct feature-classes — DERIVED, never a duplicate source per §11.4.93). The summary feeds the §11.4.132 risk-ordering inputs (fail-runs / status-changes / last-result) without re-reading every observations blob.

(c) Four-format per-item exports, docs_chain-bound, never missed. Each item's `Diary` + `Diary_Summary` render to the project's full export-format set (§11.4.65 + any project-added format) with the freshness invariant (derived-format mtime ≥ source mtime), wired through the §11.4.106 docs_chain engine as a registered consumer context with a §11.4.86 drift-proof fingerprint (sha256 of the sorted diary keyset, NOT mtime) so a diary row added but not exported/pushed FAILs a pre-build gate — the never-missed mechanism.

(d) External-tracker sub-task model with lifecycle. On the external tracker, the workable-item task keeps its description = the item (§11.4.148 D2 — NOT polluted with diary text); each diary run is a CHILD SUB-TASK of that task (type `task`), carrying a `{TODO | In-progress | Completed}` lifecycle status (collapsed onto the tracker's native status set per §11.4.33 / §11.4.112 when the tracker's vocabulary is fixed), its observations + action + an Evidence: line (the path, not the raw artefact — §11.4.10 / §11.4.13), and a stable diary-entry idempotency key. The push reuses the §11.4.148 D5 idempotent, rate-limited, credential-redacted, sink-side-proven discipline; the sub-task mapper stays project-agnostic (§11.4.28).

(e) Minimal-LLM, anti-bluff, 100% test-covered. The diary tooling (add / export / tracker-sync / validate) is deterministic bash/Go with ZERO LLM in the data path — the `observations` prose is authored by whoever ran the test; the tooling only stores/renders/pushes/validates it. The diary is covered per §11.4.27 by every applicable test type (unit + integration-against-the-real-tracker with an honest §11.4.3 SKIP-when-token-absent, never a faked PASS + export + HelixQA

Challenge + paired §1.1 mutation) so a diary PASS-bluff is mechanically impossible. Honest boundary (§11.4.6): the diary guarantees a complete, auditable test-history per item, NOT that any single run's verdict is correct — that rests on the run's own captured evidence per §11.4.69 / §11.4.107 / §11.4.123.

Classification: universal (§11.4.17) — the consuming project supplies its concrete DB path, diary doc layout, export formats, external-tracker sub-task field map, and docs_chain context per §11.4.35. Composes §11.4.6 / §11.4.27 / §11.4.45 / §11.4.50 / §11.4.55 / §11.4.65 / §11.4.69 / §11.4.86 / §11.4.93 / §11.4.95 / §11.4.106 / §11.4.107 / §11.4.123 / §11.4.132 / §11.4.148 / §11.4.10 / §11.4.13 / §11.4.28 / §11.4.33 / §11.4.112 / §1.1. Propagation gate `CM-COVENANT-114-149-PROPAGATION` (literal `11.4.149`) + recommended gates `CM-TEST-DIARY-SYNC` (the diary fingerprint gate — a diary row not exported/pushed FAILs) / `CM-DIARY-PASS-REQUIRES-EVIDENCE` (a PASS diary row carries a non-empty evidence path) + paired §1.1 meta-test mutations (strip the literal → propagation gate FAILs; strip the evidence-required constraint OR the sub-task lifecycle / fingerprint → the respective gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.149. Non-compliance is a release blocker. No escape hatch — no `--skip-testing-diary`, `--diary-without-evidence`, `--no-diary-summary`, `--diary-without-tracker-subtask`, `--llm-in-diary-data-path`, `--diary-export-optional` flag.

§11.4.171 — Mandatory comprehensive human-readable workable-item descriptions (User mandate, 2026-06-29). Every workable item (ATM-NNN ticket) in the project's workable-items database, Issues.md, Fixed.md, Issues_Summary.md, Fixed_Summary.md, and ALL derived documents MUST carry a comprehensive human-readable description that explains the item in plain, non-technical language suitable for non-developers (project managers, stakeholders, team leads, business partners). The description MUST be 5-7 sentences minimum, written so that ANY person — regardless of technical background — can understand: (a) WHAT the item is (the feature, fix, or task in simple terms); (b) WHY it matters (the user/business value); (c) HOW it works (the mechanism, without code references); (d) WHO benefits (the end user or stakeholder); (e) WHAT the expected outcome is (the measurable result). The description is MANDATORY — an item without a human-readable description is a §11.4 violation at the documentation layer. Descriptions MUST NOT use jargon,

acronyms without expansion, code references, or technical implementation details as the primary explanation. Technical details MAY appear as supplementary context AFTER the plain-language explanation. The `description` column in the SQLite database (`docs/workable_items.db`) is the SINGLE SOURCE OF TRUTH for item descriptions; all derived documents (Issues.md, Fixed.md, summaries, exports) MUST carry the SAME description text. Pre-build gate `CM-HUMAN-READABLE-DESCRIPTION` (when implemented) walks every item in the DB and asserts the `description` field exists, is non-empty, and is ≥ 50 characters. Paired §1.1 meta-test mutation strips a description → gate FAILs. Classification: universal (§11.4.17). Composes §11.4.15 (item-status tracking), §11.4.16 (item-type tracking), §11.4.19 (column alignment), §11.4.91 (summary clarity), §11.4.93 (SQLite SSoT), §11.4.148 (item integrity), §11.4.65 (universal export). Propagation gate `CM-COVENANT-114-171-PROPAGATION` (literal `11.4.171`) + paired §1.1 mutation.

§11.4.202 — Reporting directives: a report MUST auto-create a fully-populated, fully-synced workable item — never a prose acknowledgement (User mandate, 2026-07-15). Verbatim operator mandate: introduce reporting directives `ISSUE` / `BUG` / `TASK` that create a proper workable item filled with all details, with the WHOLE workable-items system automatically up to date and fully in sync — the main DB, all documents, and all external tracking systems; universal, reusable on every project, fully decoupled, recognized immediately by every project that pulls the constitution. Every governed project MUST expose REPORTING DIRECTIVES — the §11.4.140 registered actions **`BUG`** (Type=Bug), **`TASK`** (Type=Task), and **`ISSUE`** (generic report, CLASSIFIED into the §11.4.16 closed set) — such that a plain-language report from the operator is TURNED INTO a real, tracked, fully-synced workable item, automatically, every time. A report that is discussed, acknowledged, answered in prose, or "noted" WITHOUT landing a tracked item is a §11.4.202 violation of §11.4 PASS-bluff severity at the requirements-intake layer: the requirement was accepted and then silently evaporated (the §11.4.197 loss-of-requirements failure, at its entry point). **(1) GRAMMAR (§11.4.140).** The three directives are ordinary registry rows, so EVERY §11.4.140 form works out of the box (`BUG :: <text>` , `DEFAULT::BUG :: <text>` , `/DEFAULT::BUG <text>` , `BUG ---> <text>`), PLUS a SIXTH form this anchor adds — the **single-colon form** **`NAME: <text>`** (`ISSUE: subtitles are late`), the shape operators actually type. The single-colon form is **REGISTERED-ACTION-ONLY by construction**: a single-colon token

that is NOT a registered action is a NO-OP (an ordinary prompt), NEVER an ASK and NEVER a sub-system shortcut — because ordinary English prose (`TODO:` , `WARNING:` , `FIXME:`) shares that shape (`NOTE:` is now a registered §11.4.140 severity marker), and routing it to the §11.4.66 clarify path would question every such sentence (§11.4.6 honest boundary: the other five forms keep their ASK-on-unknown behaviour — their shapes do not occur in prose). Host-command collisions are handled by the EXISTING §11.4.140 conflict mechanism (`slash_bare: auto` + `slash_conflicts: [..]`), which already satisfies the operator's explicit-prefix requirement: a bare `/NAME` that collides with a built-in (e.g. the documented Claude Code built-in `/bug`) is NOT honored, while the namespaced `/DEFAULT::NAME` form is ALWAYS unambiguous — and the `NAME:` / `NAME ::` / `NAME --->` forms are never affected by any host command. **(2) THE ITEM.** The created item MUST carry, on ALL surfaces: a valid Status (§11.4.15, `Queued` at intake) + a Type from the CLOSED set {Bug | Feature | Task} (§11.4.16 — `ISSUE` is a REPORTING CHANNEL, never a fourth type; inventing one is a violation) + a stable auto-incremented id (§11.4.54), and a COMPREHENSIVE structured description (§11.4.148 D2 / §11.4.171): what / affected-scope / reproduction / acceptance. An undetermined section is recorded as an explicit `UNKNOWN:` gap — NEVER invented (§11.4.6). For `ISSUE` , the type is classified from the report's content and stated as FACT; if the content does not determine it, the operator is ASKED (§11.4.66 / §11.4.105) BEFORE creation; ONLY when running autonomously where asking is impossible (§11.4.101) may the §11.4.16 lowest-stakes ambiguity default `Task` be used — and then the defaulted classification MUST be recorded verbatim in the item and surfaced for reclassification, never silently asserted. **(3) THE FULL SYNC (the load-bearing half).** Creating the item is NOT sufficient: the directive MUST drive the WHOLE workable-items system into sync, in one shot — (a) the item lands in the SQLite single-source-of-truth (§11.4.93 / §11.4.95); (b) EVERY derived document is regenerated FROM the DB — the open + closed trackers, their summaries, and their `.md` / `.html` / `.pdf` / `.docx` siblings (§11.4.12 / §11.4.53 / §11.4.65 / §11.4.106); (c) the item is pushed to EVERY configured external tracker (§11.4.148 D5). **(4) NEVER A FAKED PUSH (§11.4.10 / §11.4.6 / §11.4.3).** A tracker whose credentials or whose client are absent is SKIPPED with an honest machine-readable reason (`credentials_absent` — names of the unset variables ONLY, never a value; `tracker_client_absent` — PENDING-OPERATOR-INPUT). A tracker PASS may be recorded ONLY after a real push command really exited 0.

Fabricating, assuming, or silently omitting a push is a §11.4 bluff at the integration layer; a missing tracker NEVER blocks the item from landing in the DB + docs (the report is never lost because a mirror is unreachable). **(5) DECOUPLED ENGINE, CONSUMER-OWNED DATA (§11.4.28 / §11.4.177).** The item-creation + full-sync engine lives in the constitution submodule (`constitution/scripts/reporting/report_item.sh`), is inherited BY REFERENCE (never copied), and carries ZERO project literals; every project-specific value — DB path, id prefix, sync command, tracker commands + their required env vars — comes from a CONSUMER-OWNED config file (DATA, never an engine edit). The engine FAILS CLOSED with an actionable message when that config is absent — it never guesses a project's paths (§11.4.6). **(6) ANTI-BLUFF (§11.4 / §11.4.107(10)).** The mechanism ships four-layer coverage: a pre-build gate (`CM-REPORTING-DIRECTIVES`) whose FUNCTIONAL invariant SOURCES and RUNS the parser (never a grep-only assertion, §11.4.108), a paired §1.1 mutation test that proves each invariant is genuinely load-bearing, and an end-to-end suite run against a REAL SQLite DB with REAL binaries (no mocks, §11.4.27) that asserts a real row lands with the right Type/Status/id, that the docs are regenerated FROM the DB, and that an absent tracker SKIPs honestly — self-validated with a golden-good + golden-bad + negative-control fixture set (an oracle that passes its golden-bad fixture is itself the bluff). Classification: universal (§11.4.17) — the consuming project supplies its DB path, id prefix, sync command, and tracker bindings per §11.4.35. Composes §11.4.140 (grammar) / §11.4.93 / §11.4.95 (DB SSoT) / §11.4.15 / §11.4.16 / §11.4.54 (status + type + id) / §11.4.148 / §11.4.171 (comprehensive description) / §11.4.106 / §11.4.12 / §11.4.53 / §11.4.65 (doc sync) / §11.4.10 (credentials) / §11.4.66 / §11.4.105 (clarify, never guess) / §11.4.101 (autonomous decision) / §11.4.197 (a started requirement never evaporates) / §11.4.28 / §11.4.177 (decoupling) / §11.4.164 (auto-registration on constitution pull) / §11.4.107(10) / §1.1. Propagation gate `CM-COVENANT-114-202-PROPAGATION` (literal `11.4.202`) + recommended gate `CM-REPORTING-DIRECTIVES` (registry declares ISSUE/BUG/TASK + the single-colon form is registered-only; the engine exists, is parse-clean, and is project-literal-free; the three directives really expand at runtime while prose does not; the honest-skip reasons exist and a tracker PASS is gated on a real exit 0) + paired §1.1 mutation (strip an action row, flip the single-colon form to non-registered-only, strip the parser branch, remove the prose NO-OP branch, make the engine fake a tracker push, or couple the engine to one project → the gate FAILs; strip the literal → the propagation gate FAILs).

Canonical authority: constitution submodule `Constitution.md` §11.4.202.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--report-without-item`, `--prose-acknowledgement-OK`, `--skip-item-creation`, `--skip-tracker-sync`, `--fake-tracker-push`, `--invent-a-fourth-type`, `--hardcode-project-in-engine` flag.

2026 07 17

§ . . — Recurrence-links-not-mints: a defect that returns MUST reopen its existing item, never enter as a new id (research-derived, - -)

Forensic anchor (genericised, 2026-07-17). In a consuming project, operator-reported defects that were RECURRENCES of already-tracked, already-"fixed" defects re-entered the tracker as BRAND-NEW ids rather than reopening the originals (measured: at least five distinct recurrence chains, one three deep). A recurrence filed as a new id **never increments the original's reopen counter** — so the §11.4.55 `reopens_count`, which is the exact signal §11.4.132(d) and §11.4.189 use to rank the most-fragile work for the deepest scrutiny, **stays quiet precisely on the items that keep breaking**. The project's recorded reopen-per-fix rate was 52% (published elite <10%) and was simultaneously a KNOWN UNDERCOUNT for this reason: the metric that was supposed to find the fragile items was silenced by the intake path. The same defect inflates the queue and forks the work — in one measured window ~15 tracker items mapped to ~9 real root causes, so "fix all 15" would have produced duplicate and mutually-conflicting fixes on one cause.

The mandate. Before minting a NEW id for any incoming defect — from ANY intake path — the system MUST check whether an existing item already describes the SAME defect, and if so LINK the report to that item rather than mint a new id, and REOPEN that item **if and only if it is currently in a terminal state** (§11.4.33 `Fixed` / `Implemented` / `Completed`, or a §11.4.90 closure OTHER than `duplicate-of` — see the chain rule).

(0) RESOLVE TO THE CANONICAL CHAIN HEAD FIRST — a `duplicate-of` closure is a POINTER, not a destination. §11.4.90's closure vocabulary includes `duplicate-of`, and clause (5) below MANUFACTURES exactly such items; the (2) key will match them, because a duplicate shares its original's subject BY

CONSTRUCTION. Therefore, before applying link-and-iff-terminal-reopen, every match MUST be resolved THROUGH its `duplicate-of` links to the CANONICAL CHAIN HEAD (the item the chain ultimately points to), and the link + any reopen land on THAT head — never on an intermediate duplicate. Reopening a `duplicate-of` closure would resurrect a non-canonical copy alongside its still-live canonical item: two live items for one defect — the exact queue-inflation + forked-fixes disease this anchor's own forensic names — and it would credit the §11.4.55 reopens signal to the WRONG item, defeating clause (6), which IS the point, on precisely the chain-class items the anchor was minted for. **Multi-match resolution:** when the key matches several items (an original plus its duplicates, possibly plus an open item), resolve every match to its chain head, then act on the SINGLE surviving head; if that resolution yields more than one distinct head the verdict is UNDECIDED per clause (3) (ask, or mint-with-link — never guess which head is meant, §11.4.6); if the resolved head is OPEN, prefer it and LINK ONLY. A chain whose head cannot be resolved (a broken/circular `duplicate-of` link) is UNDECIDED + a tracked integrity defect (§11.4.186 orphan-ref class), never a silent pick.

Open-original case (the ordinary two-reports-of-one-live-bug): when the resolved chain head is still OPEN — any non-terminal §11.4.15 status, i.e. `Queued` / `In progress` / `Ready for testing` / `In testing` / already `Reopened` / `Operator-blocked` (§11.4.21's 7th value, non-terminal) — the correct action is LINK ONLY — "reopen an open item" is undefined in the §11.4.15 status machine, and §11.4.34's `Reopened` status + its reason vocabulary presuppose a prior closure to demote FROM (§11.4.7); forcing a reopen there would corrupt both the status and the counter. The reopen-and-increment path exists to correct the specific lie "this was closed as fixed and it is not" — an already-open item is not telling that lie.

§11.4.202 precedence (stated explicitly — never left for a consumer to reconcile). §11.4.202 mandates that every report lands a tracked item, "automatically, every time", and forbids `--skip-item-creation`; §11.4.214 forbids `--mint-every-report`. These are NOT in conflict and MUST NOT be read as such: §11.4.202's requirement is that **no report is ever LOST** — it is satisfied IN FULL by a SAME-DEFECT report landing as a link + (terminal-only) reopen on the existing item, because the report is tracked, the item is live, and the §11.4.55 counter now tells the truth. §11.4.202 forbids the report evaporating into prose; it does NOT require a fresh id per report. §11.4.214 refines WHICH item a

report lands on; §11.4.202 owns THAT it lands. Where the check verdict is DISTINCT or UNDECIDED, §11.4.202's mint-a-new-id path runs unchanged. **(1) EVERY INTAKE PATH IS BOUND, not merely the reporting channel.** Defects enter via the §11.4.202 reporting directives, via AI-detected test/gate failures, via regression-guard verdicts, via cycle re-discovery, via captured-evidence retrospect, and via operator manual testing — §11.4.34's own reason vocabulary (`test-failed` , `cycle-re-discovered` , `manual-testing-detected` , `end-user-report`) enumerates them. A dedup check wired ONLY into the reporting channel under-covers exactly the machine-driven paths that recur most. **(2) THE KEY IS STRUCTURED, NEVER A BARE SUBSTRING (§11.4.186).** Match on ticket reference where present, else on a normalised `(subject, scope)` — explicitly NOT a bare subject substring, which is the false-merge generator. **(3) A WRONG MERGE SILENTLY DELETES A REAL DEFECT — so `distinct-but-similar` is a FIRST-CLASS VERDICT, not a failure.** The check has three outcomes: SAME-DEFECT (link + reopen), DISTINCT (mint a new id — correct and expected), and UNDECIDED (§11.4.6: do NOT guess — ask per §11.4.66/§11.4.105, or mint the new id WITH a recorded candidate-duplicate link for review; when autonomous per §11.4.101, prefer minting-with-link over merging, because a spurious extra id is cheap and recoverable while a wrongly-merged defect is silently LOST — the asymmetry decides the default). **(4) THE DEDUP ORACLE IS SELF-VALIDATED (§11.4.107(10) / §11.4.186).** It ships a golden-good (a true recurrence — MUST link+reopen), a golden-bad (a missed recurrence — MUST be caught), and a **negative-control**: two genuinely DISTINCT items that merely share a subject, which MUST NOT be merged. An oracle without the negative control is a false-merge engine and is itself a §11.4 bluff. **(5) ID STABILITY IS PRESERVED (§11.4.54).** Linking a recurrence to its original NEVER renumbers, reuses, or retires an id: the original is REOPENED and its counter increments; an already-minted duplicate is LINKED to the original (a recorded duplicate-of relation per §11.4.90) — never silently deleted, never renumbered. **(6) THE RESTORED SIGNAL IS THE POINT.** Reopening the original is what makes §11.4.55's counter true, which is what makes §11.4.132(d)/§11.4.189 rank the genuinely-fragile items first. A reopen counter that undercounts is not a cosmetic defect — it is a §11.4/§11.4.1 bluff at the prioritisation layer: it reports the fragile set as stable, and the deepest scrutiny is then aimed at the wrong items.

Honest boundary (§11.4.6). Dedup-at-intake makes the reopen signal TRUE; it does not make the item CORRECT, nor prove the recurrence has the same root cause as the original (a same-symptom recurrence with a genuinely different cause is a real possibility — §11.4.102 systematic-debugging decides that, and the linked-and-reopened item is where that investigation is recorded, not a reason to fork a new id). This anchor BINDS rather than re-authors: the intake seam is §11.4.202's, the keying rule and negative-control discipline are §11.4.186's, the reopen mechanics and attribution are §11.4.34's, the counter and its history document are §11.4.55's, id stability is §11.4.54's, and the priority consumers are §11.4.132/ §11.4.189.

Classification: universal (§11.4.17) — references no project-specific hardware, vendor, or tracker; the consuming project supplies its tracker, its id scheme, its normalisation function, and its intake wiring per §11.4.35. Composes §11.4.1 / §11.4.6 / §11.4.34 / §11.4.54 / §11.4.55 / §11.4.66 / §11.4.90 / §11.4.93 / §11.4.101 / §11.4.102 / §11.4.105 / §11.4.107(10) / §11.4.132 / §11.4.148 / §11.4.186 / §11.4.189 / §11.4.202. Propagation gate `CM-COVENANT-114-214-PROPAGATION` (literal `11.4.214`) + recommended gate `CM-RECURRENCE-LINKS-NOT-MINTS` (every intake path runs the dedup check before minting; the oracle's golden-good/ golden-bad/negative-control are wired into the meta-test; every CONFIRMED-duplicate link resolves to a canonical CHAIN HEAD, and every link whose resolved head was in a TERMINAL state at link time has a corresponding reopen event ON THAT HEAD). **The gate's own scope is load-bearing and is itself §11.4.201-bound:** it MUST NOT fire on a link whose resolved head was OPEN at link time (LINK-only is the CORRECT action there per the open-original case), nor on a clause-(3) UNDECIDED `candidate-duplicate` link (which is by design un-reopened, and is the mandated autonomous default), nor demand a reopen on an intermediate `duplicate-of` closure (clause (0) FORBIDS that reopen — a gate demanding it would enforce the very defect the anchor prevents) — a gate firing on any of these would be exactly the §11.4.201(1) false-positive refusal that halts correct work and teaches operators to bypass it, and it would condemn behaviour THIS anchor mandates. Its golden-FALSE fixture set MUST therefore include an open-head link, a candidate-duplicate link, AND a link whose immediate target is a `duplicate-of` closure with the reopen correctly landing on the chain head — the gate MUST NOT fire on any of the three (§11.4.201(3) / §11.4.107(10)); its golden-TRUE set MUST include a terminal-head link with NO reopen event (the gate MUST fire) (file a known recurrence and let it mint a fresh id without a link → the

gate FAILs; strip the negative control so two distinct same-subject items merge → the oracle self-check FAILs; strip the literal → the propagation gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.214.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--skip-dedup-at-intake`, `--mint-every-report`, `--bare-substring-dedup`, `--merge-without-negative-control`, `--recurrence-as-new-id-OK`, `--reopen-counter-may-undercount` flag exists.